

Intervals & Sweep Line Algorithms

MERGE INTERVALS

1. Overview

Merge Intervals is a classic problem where you are given a collection of intervals `[start, end]` and you need to merge all overlapping intervals into a single consolidated set. Two intervals overlap if one starts before or at the same time the other ends.

Example: `[[1,3], [2,6], [8,10], [15,18]]` $\rightarrow$ `[[1,6], [8,10], [15,18]]`

2. Intuition

Think of intervals as time slots on a timeline. If two meetings overlap in time, you can merge them into one longer meeting that covers the entire duration.

Step-by-step reasoning:

- 1. First, sort all intervals by their start time. This ensures we process them in chronological order.
- 2. Start with the first interval as your current "merged" interval.
- 3. For each subsequent interval:
 - If it overlaps with the current merged interval (its start $\leq$ current end), extend the current end to `max(current.end, interval.end)`.
 - If it does not overlap, save the current merged interval and start a new one.

Why this works: After sorting by start time, overlapping intervals are guaranteed to be adjacent in the sorted order (or nested within earlier ones). We only need to check the last merged interval's end against the next interval's start.

3. When to Use It

Situation	Example Problem Phrase
Given a list of intervals, merge overlapping ones	"Merge all overlapping intervals"
Count number of intervals after merging	"Find the minimum number of intervals to cover all points"
Check if intervals overlap	"Determine if there is any overlap"
Find free time slots	"Find free time in a schedule"
Combine ranges	"Merge overlapping ranges"

Trigger phrases: "overlapping intervals", "merge intervals", "consolidate ranges", "remove overlaps", "coverage"

4. When Not to Use It

- **When intervals are already non-overlapping:** No merging needed; simple iteration suffices.
- **When you need point queries, not range merging:** Use difference array or sweep line instead.
- **When intervals are extremely large and sorting is expensive:** If you need real-time insertion, use an interval tree or segment tree.
- **When you need to query arbitrary overlaps efficiently:** Use an interval tree ($O(\log n)$ per query) instead of $O(n)$ after merging.

5. Core Concepts

5.1 Overlap Condition

Two intervals $[a, b]$ and $[c, d]$ overlap if $c \leq b$ (assuming sorted by start). They do NOT overlap if $c > b$.

5.2 Merging

When two intervals overlap, the merged interval is $[\min(\text{start1}, \text{start2}), \max(\text{end1}, \text{end2})]$.

5.3 Sorting by Start

The key preprocessing step. Sorting by start time puts intervals in order so we only need a single pass.

5.4 Result Construction

We build the result incrementally, always keeping a "current" interval that we try to extend.

6. Step-by-Step Algorithm

```
Input: intervals[][] = [[s1,e1], [s2,e2], ...]

1. If intervals is empty, return empty list.
2. Sort intervals by start time (ascending).
3. Initialize result = [intervals[0]].
4. For each interval [s, e] in intervals[1:]:
   a. Let last = last interval in result.
   b. If s <= last.end:           // overlap
       last.end = max(last.end, e) // merge
   c. Else:                       // no overlap
       result.push([s, e])         // add new interval
5. Return result.
```

7. Dry Run

Input: $[[1,3], [2,6], [8,10], [15,18], [17,20]]$

Step 1: Sort by start (already sorted).

Step	Current Interval	Last in Result	Overlap?	Result
Init	—	—	—	[[1,3]]
1	[2,6]	[1,3]	Yes ($2 \leq 3$)	[[1,6]]
2	[8,10]	[1,6]	No ($8 > 6$)	[[1,6], [8,10]]
3	[15,18]	[8,10]	No ($15 > 10$)	[[1,6], [8,10], [15,18]]
4	[17,20]	[15,18]	Yes ($17 \leq 18$)	[[1,6], [8,10], [15,20]]

Final Answer: [[1,6], [8,10], [15,20]]

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

vector<vector<int>> mergeIntervals(vector<vector<int>>& intervals) {
    if (intervals.empty()) return {};

    // Step 1: Sort by start time
    sort(intervals.begin(), intervals.end());

    vector<vector<int>> result;
    result.push_back(intervals[0]);

    for (int i = 1; i < (int)intervals.size(); i++) {
        int s = intervals[i][0];
        int e = intervals[i][1];
        vector<int>& last = result.back();

        if (s <= last[1]) {
            // Overlap – merge by extending end
            last[1] = max(last[1], e);
        } else {
            // No overlap – add new interval
            result.push_back({s, e});
        }
    }

    return result;
}

// Example usage
int main() {
    vector<vector<int>> intervals = {{1,3}, {2,6}, {8,10}, {15,18}};
    auto merged = mergeIntervals(intervals);

    cout << "Merged intervals: ";
    for (auto& iv : merged) {
        cout << "[" << iv[0] << "," << iv[1] << "]" << " ";
    }
}
```

```

    }
    // Output: [1,6] [8,10] [15,18]

    return 0;
}

```

9. Python Implementation

```

def merge_intervals(intervals):
    if not intervals:
        return []

    # Step 1: Sort by start time
    intervals.sort(key=lambda x: x[0])

    result = [intervals[0]]

    for s, e in intervals[1:]:
        last_s, last_e = result[-1]

        if s <= last_e:
            # Overlap – merge
            result[-1][1] = max(last_e, e)
        else:
            # No overlap – add new
            result.append([s, e])

    return result

# Example usage
if __name__ == "__main__":
    intervals = [[1,3], [2,6], [8,10], [15,18]]
    merged = merge_intervals(intervals)
    print(f"Merged intervals: {merged}")
    # Output: [[1, 6], [8, 10], [15, 18]]

```

10. Code Explanation

- **Sorting:** `sort(intervals.begin(), intervals.end())` sorts by the first element (start time) ascending. This is the critical preparation step.
- **Result initialization:** We push the first interval to start the chain.
- **Loop:** For each interval, we check if it overlaps with the last interval in the result.
- **Overlap check:** `s <= last[1]` — the current start is before or at the last interval's end. This is the key condition.
- **Merge:** `last[1] = max(last[1], e)` — we extend the end to cover both intervals. We use `max` because the current interval might end before `last[1]` (nested intervals).
- **No overlap:** We push the current interval as a new entry.

Edge case handling: `if (intervals.empty()) return {}` handles empty input.

11. Complexity Analysis

Operation	Time	Space
Sorting	$O(n \log n)$	$O(\log n)$ (sorting stack)
Merging pass	$O(n)$	$O(1)$ extra
Overall	$O(n \log n)$	$O(n)$ (for result, or $O(1)$ if in-place)

- **Best case:** Already sorted intervals — still $O(n \log n)$ due to sort.
- **Worst case:** No merges — same complexity.
- **Space:** $O(n)$ to store the result in worst case (no merges). If we modify in-place, $O(1)$ extra.

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Merge all overlaps	"Merge overlapping intervals"	Sort + single pass merge	Merge Intervals (LC 56)
Count non-overlapping	"Minimum number of intervals to remove"	Sort by end, greedy	Non-overlapping Intervals (LC 435)
Check if any overlap	"Can attend all meetings"	Sort, check adjacent	Meeting Rooms (LC 252)
Find gaps	"Free time slots", "Available time"	Merge then check gaps	Employee Free Time (LC 759)
Insert and merge	"Insert a new interval"	Binary search + merge	Insert Interval (LC 57)

13. Common Mistakes

- **Not sorting:** Without sorting, you cannot guarantee that overlaps are detected in one pass.
- **Wrong overlap condition:** Using `s < last[1]` instead of `s <= last[1]`. If one interval ends exactly where another starts, they are not overlapping (unless the problem says otherwise).
- **Not using `max` on end:** For nested intervals like `[1,5]` and `[2,3]`, using `last[1] = e` would shrink the interval to `[1,3]`. Always use `max(last[1], e)`.
- **Modifying input:** Sorting the original array modifies it. Make a copy if the original is needed later.
- **Integer overflow:** For large values (e.g., 10^9), `int` is fine, but use `long long` if needed.

14. Edge Cases

Input	Expected	Notes
<code>[]</code>	<code>[]</code>	Empty input
<code>[[1,2]]</code>	<code>[[1,2]]</code>	Single interval

Input	Expected	Notes
[[1,5], [2,3], [4,6]]	[[1,6]]	Nested + overlapping
[[1,2], [3,4], [5,6]]	[[1,2], [3,4], [5,6]]	No overlaps
[[1,10], [2,3], [4,5]]	[[1,10]]	All contained
[[1,4], [0,2], [3,5]]	[[0,5]]	Unsorted input
[[1,4], [4,5]]	[[1,5]]	Touch at endpoint (overlap by $\leq$)

15. Variations

15.1 In-place Merge

Modify the input array directly instead of building a new result. Saves space but mutates input.

15.2 Merge with Custom Comparator

Sometimes intervals are [end, start] or have other structures. Write a custom comparator for sorting.

15.3 Merge K Lists of Intervals

Merge intervals from multiple sorted lists. Use a priority queue (min-heap) to merge efficiently.

15.4 Merge with Gaps

Merge intervals but also track gaps (free time). Useful for calendar/availability problems.

16. Related Algorithms/Data Structures

Algorithm	Connection	When to Choose
Sweep Line	Processes events at points rather than merging ranges	When you need point queries or counts
Interval Tree	BST of intervals for overlap queries	When you need dynamic insertion/deletion/query
Segment Tree	Range query and update	When you need to query arbitrary ranges with updates
Difference Array	Range addition	When you need to add values to ranges and query a point

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Merge Intervals	LeetCode 56	Basic merge	Easy

Problem	Platform	Pattern	Difficulty
Meeting Rooms	LeetCode 252	Check overlap	Easy

Medium

Problem	Platform	Pattern	Difficulty
Insert Interval	LeetCode 57	Insert + merge	Medium
Non-overlapping Intervals	LeetCode 435	Greedy removal	Medium
Minimum Number of Arrows to Burst Balloons	LeetCode 452	Sort by end	Medium

Hard

Problem	Platform	Pattern	Difficulty
Employee Free Time	LeetCode 759	Merge + gaps	Hard
Data Stream as Disjoint Intervals	LeetCode 352	Dynamic insertion	Hard
Range Module	LeetCode 715	Add/remove/query ranges	Hard

18. Interview Explanation

"Merge Intervals is a sorting-based problem. The key insight is that after sorting by start time, any overlapping intervals will be adjacent in the sorted order. I maintain a result list and keep a 'current' interval. For each new interval, if it overlaps with the current one ($\text{start} \leq \text{current_end}$), I merge by extending the end to the maximum of both ends. If it doesn't overlap, I push the current interval to result and start a new one. This gives $O(n \log n)$ time due to sorting and $O(n)$ space for the result."

19. Revision Notes

- **Sort by start** — always the first step.
- **Overlap condition:** $\text{new_start} \leq \text{current_end}$
- **Merge:** $\text{current_end} = \max(\text{current_end}, \text{new_end})$
- **No overlap:** Push current, start new.
- **Nested intervals:** Always use $\max$ on end.
- **Edge cases:** Empty, single, sorted, touching at endpoint.
- **Complexity:** $O(n \log n)$ time, $O(n)$ space.

20. Final Cheat Sheet

Aspect	Details
When to use	Overlapping ranges, schedule consolidation, coverage problems
Main operations	Sort by start, one-pass merge
Time	$O(n \log n)$

Aspect	Details
Space	O(n) result or O(1) in-place
Key code	<pre>if (s <= last[1]) last[1] = max(last[1], e); else result.push_back({s, e});</pre>
Edge cases	Empty, nested intervals, touching at endpoint
Common trap	Forgetting <code>max()</code> on merge, wrong overlap condition, not sorting

INSERT INTERVAL

1. Overview

Insert Interval is a problem where you are given a set of non-overlapping intervals sorted by start time, and you need to insert a new interval into them, merging if necessary. The input intervals are already sorted and non-overlapping.

Example: `Intervals = [[1,3], [6,9]], New = [2,5] → [[1,5], [6,9]]`

2. Intuition

Think of inserting a new meeting into an existing calendar that has no overlapping meetings. You find where the new meeting fits, merge it with any meetings it overlaps, and keep the rest unchanged.

Step-by-step reasoning:

1. Since intervals are already sorted, we can process them in order.
2. There are three phases:
 - **Before overlap:** Intervals that end before the new interval starts — add them directly.
 - **During overlap:** Intervals that overlap with the new interval — merge them into one.
 - **After overlap:** Intervals that start after the merged interval ends — add them directly.

Why this works: The sorted property guarantees that once we pass the overlap region, no future interval can overlap with the merged interval.

3. When to Use It

Situation	Example Problem Phrase
Insert one interval into a sorted non-overlapping list	"Insert a new interval into the schedule"
Add a range to a set of ranges	"Add a new range to existing ranges"
Schedule a new meeting	"Insert a meeting into calendar"

Trigger phrases: "insert interval", "add interval", "schedule meeting", "non-overlapping sorted intervals"

4. When Not to Use It

- **When intervals are not sorted:** Sort first, or use the Merge Intervals approach (sort + merge after adding).
- **When you need to insert many intervals dynamically:** Use an interval tree or segment tree for $O(\log n)$ per insertion.
- **When you need to check if insertion is possible without merging:** Just check for overlap, don't need the full algorithm.

5. Core Concepts

5.1 Three Phases

The algorithm naturally divides into three parts: before overlap, during overlap, after overlap.

5.2 Merging During Overlap

For overlapping intervals, the merged interval is $[\min(\text{newStart}, \text{existingStart}), \max(\text{newEnd}, \text{existingEnd})]$.

5.3 Early Break

Once the new interval is placed, we can add all remaining intervals unchanged.

6. Step-by-Step Algorithm

Input: `intervals[][]` (sorted, non-overlapping), `newInterval [s, e]`

```

1. Initialize result = [].
2. i = 0.
3. // Phase 1: Add all intervals ending before newInterval starts
   While i < n and intervals[i][1] < s:
       result.push(intervals[i])
       i++

4. // Phase 2: Merge overlapping intervals
   While i < n and intervals[i][0] <= e:
       s = min(s, intervals[i][0])
       e = max(e, intervals[i][1])
       i++
   result.push([s, e])

5. // Phase 3: Add remaining intervals
   While i < n:
       result.push(intervals[i])
       i++

6. Return result.
```

7. Dry Run

Input: `Intervals = [[1,2], [3,5], [6,7], [8,10], [12,16]]`, `New = [4,8]`

Phase	i	Current Interval	New Interval	Action	Result
1	0	[1,2] — end $2 < 4$	—	Add directly	[[1,2]]
1	1	[3,5] — end $5 \geq 4$	—	Stop phase 1	—
2	1	[3,5] — start $3 \leq 8$	Merge $\rightarrow$ [3,8]	Merge	—
2	2	[6,7] — start $6 \leq 8$	Merge $\rightarrow$ [3,8]	Merge	—
2	3	[8,10] — start $8 \leq 8$	Merge $\rightarrow$ [3,10]	Merge	—
2	4	[12,16] — start $12 > 8$	—	Stop phase 2	[[1,2], [3,10]]
3	4	[12,16]	—	Add directly	[[1,2], [3,10], [12,16]]

Final Answer: [[1,2], [3,10], [12,16]]

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

vector<vector<int>> insertInterval(vector<vector<int>>& intervals,
                                vector<int>& newInterval) {
    vector<vector<int>> result;
    int n = intervals.size();
    int i = 0;
    int s = newInterval[0], e = newInterval[1];

    // Phase 1: Intervals ending before new interval starts
    while (i < n && intervals[i][1] < s) {
        result.push_back(intervals[i]);
        i++;
    }

    // Phase 2: Merge overlapping intervals
    while (i < n && intervals[i][0] <= e) {
        s = min(s, intervals[i][0]);
        e = max(e, intervals[i][1]);
        i++;
    }
    result.push_back({s, e});

    // Phase 3: Remaining intervals
    while (i < n) {
        result.push_back(intervals[i]);
        i++;
    }
}
```

```

        return result;
    }

    // Example usage
    int main() {
        vector<vector<int>> intervals = {{1,2}, {3,5}, {6,7}, {8,10}, {12,16}};
        vector<int> newInterval = {4, 8};

        auto result = insertInterval(intervals, newInterval);

        cout << "Result: ";
        for (auto& iv : result) {
            cout << "[" << iv[0] << "," << iv[1] << "]" ";
        }
        // Output: [1,2] [3,10] [12,16]

        return 0;
    }

```

9. Python Implementation

```

def insert_interval(intervals, new_interval):
    result = []
    n = len(intervals)
    i = 0
    s, e = new_interval

    # Phase 1: Intervals ending before new interval starts
    while i < n and intervals[i][1] < s:
        result.append(intervals[i])
        i += 1

    # Phase 2: Merge overlapping intervals
    while i < n and intervals[i][0] <= e:
        s = min(s, intervals[i][0])
        e = max(e, intervals[i][1])
        i += 1
    result.append([s, e])

    # Phase 3: Remaining intervals
    while i < n:
        result.append(intervals[i])
        i += 1

    return result

# Example usage
if __name__ == "__main__":
    intervals = [[1,2], [3,5], [6,7], [8,10], [12,16]]

```

```

new_interval = [4, 8]
result = insert_interval(intervals, new_interval)
print(f"Result: {result}")
# Output: [[1, 2], [3, 10], [12, 16]]

```

10. Code Explanation

- **Phase 1 (Before overlap):** `intervals[i][1] < s` — intervals that end strictly before the new interval starts. These are unaffected and added directly.
- **Phase 2 (Overlap):** `intervals[i][0] <= e` — intervals whose start is within the new interval's range. We expand the new interval to cover all of them. After the loop, we push the merged result.
- **Phase 3 (After overlap):** All remaining intervals start after the merged interval ends. They are added as-is.
- **Variable reuse:** `s` and `e` are updated in-place to represent the merged interval.

Edge case handling: If the new interval goes before all, Phase 1 is skipped, Phase 2 merges nothing, and the new interval is inserted at the beginning. If it goes after all, Phase 1 adds everything, Phase 2 merges nothing, and the new interval is appended.

11. Complexity Analysis

Operation	Time	Space
Single pass	$O(n)$	$O(1)$
Result storage	$O(1)$	$O(n)$
Overall	$O(n)$	$O(n)$

- **Time:** $O(n)$ — we process each interval exactly once.
- **Space:** $O(n)$ for the result. $O(1)$ extra if we ignore result storage.

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Insert one interval	"Insert new interval", sorted input	Three-phase approach	Insert Interval (LC 57)
Insert with overlap check	"Can you schedule?"	Check overlap before inserting	Meeting Rooms II
Batch insert	Multiple insertions	Sort all + merge	Merge Intervals variant

13. Common Mistakes

- **Not handling empty intervals list:** If `intervals` is empty, the result should just contain `newInterval`.
- **Wrong overlap condition in Phase 2:** Using `s < intervals[i][1]` instead of `intervals[i][0] <= e`. The condition `intervals[i][0] <= e` correctly captures all overlapping cases.

- **Forgetting to update s :** The merged start should be $\min(s, \text{intervals}[i][0])$, not just s .
- **Incorrect Phase 1 condition:** $\text{intervals}[i][1] < s$ (strictly less) — intervals that end exactly at s do NOT overlap, so they go to Phase 1.
- **Modifying the input:** The new interval variable is mutated; if needed later, make a copy.

14. Edge Cases

Input	Expected	Notes
<code>[], [5,7]</code>	<code>[[5,7]]</code>	Empty intervals
<code>[[1,5]], [2,3]</code>	<code>[[1,5]]</code>	New interval is contained
<code>[[1,5]], [6,8]</code>	<code>[[1,5], [6,8]]</code>	No overlap
<code>[[1,5]], [0,0]</code>	<code>[[0,0], [1,5]]</code>	New interval before all
<code>[[1,5]], [10,12]</code>	<code>[[1,5], [10,12]]</code>	New interval after all
<code>[[1,2], [3,5]], [0,6]</code>	<code>[[0,6]]</code>	New interval covers all
<code>[[1,5]], [5,7]</code>	<code>[[1,7]]</code>	Touching at endpoint

15. Variations

15.1 Insert Without Merging (Overlap Check Only)

Only check if insertion is possible without overlap. Return boolean or throw error.

15.2 Insert into Unsorted Intervals

Sort first, then use the same algorithm. Or just append and use Merge Intervals.

15.3 Range Add (Difference Array)

When you need to add values to a range rather than merge intervals, use difference array.

16. Related Algorithms/Data Structures

Algorithm	Connection	When to Choose
Merge Intervals	Same core idea but no sorted input guarantee	When input is unsorted
Sweep Line	Processes events at start/end points	When you need point queries or counts
Interval Tree	Dynamic insertion and overlap queries	When you need frequent insertions

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Insert Interval	LeetCode 57	Three-phase insert	Medium (often classified Easy)
Meeting Rooms	LeetCode 252	Overlap check	Easy

Medium

Problem	Platform	Pattern	Difficulty
Range Module	LeetCode 715	Add/remove/query ranges	Hard
My Calendar I	LeetCode 729	Book without overlap	Medium

Hard

Problem	Platform	Pattern	Difficulty
My Calendar II	LeetCode 731	Double booking allowed	Medium
My Calendar III	LeetCode 732	Max overlap count	Hard

18. Interview Explanation

"Insert Interval leverages the fact that the input intervals are already sorted and non-overlapping. I process intervals in three phases: those that end before the new interval starts are added directly; those that overlap are merged into the new interval by expanding its start and end; those that start after the merged interval are added directly. This is a single $O(n)$ pass with $O(n)$ space for the result."

19. Revision Notes

- **Input is sorted + non-overlapping** — this is guaranteed.
- **Three phases:** before (add), during (merge), after (add).
- **Phase 1 condition:** `intervals[i][1] < newStart`
- **Phase 2 condition:** `intervals[i][0] <= newEnd`
- **Merge:** `newStart = min(newStart, intervals[i][0]), newEnd = max(newEnd, intervals[i][1])`
- **Complexity:** $O(n)$ time, $O(n)$ space.

20. Final Cheat Sheet

Aspect	Details
When to use	Insert into sorted non-overlapping intervals
Main operations	Three-phase pass
Time	$O(n)$
Space	$O(n)$
Key code	Three while loops: before, during, after

Aspect	Details
Edge cases	Empty intervals, new before all, new after all, new covers all
Common trap	Wrong overlap condition, forgetting <code>min</code> on start

NON-OVERLAPPING INTERVALS

1. Overview

Non-overlapping Intervals (also called "Maximum Non-overlapping Intervals") asks: Given a set of intervals, find the minimum number of intervals to remove so that the remaining intervals are non-overlapping. This is equivalent to finding the maximum number of non-overlapping intervals you can keep.

Example: `[[1,2], [2,3], [3,4], [1,3]]` → Remove 1 interval (either `[1,3]` or one of the others). Answer = 1.

2. Intuition

Think of it as scheduling the maximum number of non-conflicting meetings in a room. You want to pack as many meetings as possible without overlap.

Greedy choice: Always pick the meeting that ends earliest. This leaves the most room for remaining meetings.

Why this works: The Earliest Finish Time (EFT) strategy is optimal for interval scheduling. By choosing intervals that end earliest, we maximize the remaining time for other intervals. This is a classic greedy algorithm proof: if an optimal solution doesn't include the earliest-finishing interval, we can swap it in without reducing the count.

3. When to Use It

Situation	Example Problem Phrase
Find minimum removals to make intervals non-overlapping	"Minimum intervals to remove"
Find maximum number of non-overlapping intervals	"Maximum number of meetings"
Maximize resource usage without conflicts	"Maximum number of events you can attend"
Schedule maximum jobs/tasks	"Maximum number of tasks"

Trigger phrases: "non-overlapping", "minimum removals", "maximum number of intervals", "can attend most", "schedule maximum"

4. When Not to Use It

- **When intervals have weights:** If intervals have values/weights, this greedy fails. Use weighted interval scheduling (DP) instead.

- **When you need to minimize total removed length, not count:** Different problem; may need DP.
- **When intervals are very long and you need to track overlaps at points:** Use sweep line to count overlaps instead.
- **When input is dynamic:** This is a static algorithm; for dynamic insertions, use an interval tree.

5. Core Concepts

5.1 Earliest Finish Time (EFT)

The greedy strategy of picking the interval with the smallest end time. This is optimal for unweighted interval scheduling.

5.2 Problem Equivalence

"Minimum removals" = total intervals - "maximum non-overlapping intervals we can keep".

5.3 Sorting by End

Sorting by end time is the key to the greedy algorithm. Sorting by start time would give a different (incorrect) result.

5.4 Greedy Proof

The greedy choice property holds because the earliest finishing interval leaves the most remaining time, and any optimal solution can be transformed to include the earliest-finishing interval.

6. Step-by-Step Algorithm

```
Input: intervals[][]

1. If intervals is empty, return 0.
2. Sort intervals by end time (ascending).
3. Initialize count = 1 (keep first interval), lastEnd = intervals[0][1].
4. For each interval [s, e] in intervals[1:]:
    a. If s >= lastEnd:           // no overlap
        count++                  // keep this interval
        lastEnd = e              // update last end
5. Answer = n - count.
```

7. Dry Run

Input: `[[1,2], [2,3], [3,4], [1,3]]`

Step 1: Sort by end time.

Sorted	[1,2]	[2,3]	[1,3]	[3,4]
End	2	3	3	4

Step 2: Greedy selection.

i	Interval	s ≥ lastEnd?	lastEnd	Kept Count
0	[1,2]	—	2	1
1	[2,3]	2 ≥ 2 ✓	3	2
2	[1,3]	1 ≥ 3 ✗	3	2
3	[3,4]	3 ≥ 3 ✓	4	3

Kept: 3 intervals. **Removed:** 4 - 3 = 1.

Final Answer: 1

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

int eraseOverlapIntervals(vector<vector<int>>& intervals) {
    int n = intervals.size();
    if (n == 0) return 0;

    // Sort by end time
    sort(intervals.begin(), intervals.end(),
        [](const vector<int>& a, const vector<int>& b) {
            return a[1] < b[1];
        });

    int kept = 1; // Keep the first interval
    int lastEnd = intervals[0][1];

    for (int i = 1; i < n; i++) {
        int s = intervals[i][0];
        int e = intervals[i][1];

        if (s >= lastEnd) {
            // No overlap - keep this interval
            kept++;
            lastEnd = e;
        }
        // If overlap, we skip (remove) this interval
    }

    return n - kept; // Minimum removals
}

// Example usage
int main() {
    vector<vector<int>> intervals = {{1,2}, {2,3}, {3,4}, {1,3}};
    cout << "Minimum removals: " << eraseOverlapIntervals(intervals) << endl;
    // Output: 1
}
```

```
    return 0;
}
```

9. Python Implementation

```
def erase_overlap_intervals(intervals):
    n = len(intervals)
    if n == 0:
        return 0

    # Sort by end time
    intervals.sort(key=lambda x: x[1])

    kept = 1
    last_end = intervals[0][1]

    for s, e in intervals[1:]:
        if s >= last_end:
            kept += 1
            last_end = e

    return n - kept

# Example usage
if __name__ == "__main__":
    intervals = [[1,2], [2,3], [3,4], [1,3]]
    print(f"Minimum removals: {erase_overlap_intervals(intervals)}")
    # Output: 1
```

10. Code Explanation

- **Sorting:** `sort(intervals.begin(), intervals.end(), [](a, b) { return a[1] < b[1]; })` — we sort by end time ascending. This is the critical greedy step.
- **Initialization:** `kept = 1` (we always keep the first interval after sorting). `lastEnd = intervals[0][1]`.
- **Loop:** For each interval, if `s >= lastEnd`, there's no overlap, so we keep it and update `lastEnd`. If `s < lastEnd`, we skip it (count as removed).
- **Overlap condition:** `s >= lastEnd` — note the `>=`. If one interval ends exactly where another starts, they don't overlap and both can be kept.
- **Answer:** `n - kept` — minimum removals = total - maximum kept.

11. Complexity Analysis

Operation	Time	Space
Sorting	$O(n \log n)$	$O(\log n)$

Operation	Time	Space
Greedy pass	$O(n)$	$O(1)$
Overall	$O(n \log n)$	$O(\log n)$

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Max non-overlapping intervals	"Maximum number of intervals"	Sort by end, greedy	Non-overlapping Intervals (LC 435)
Minimum arrows to burst balloons	"Minimum arrows to burst"	Sort by end, greedy	Min Arrows (LC 452)
Maximum meetings	"Maximum meetings in one room"	Sort by end, greedy	N meetings in one room (GFG)
Max chain length	"Maximum length of pair chain"	Sort by end, greedy	Maximum Length of Pair Chain (LC 646)

13. Common Mistakes

- **Sorting by start instead of end:** Sorting by start and then greedily picking gives suboptimal results. Example: `[[1,5], [2,3], [3,4]]` — sorted by start gives `[1,5]` (kept=1), but optimal is `[2,3], [3,4]` (kept=2).
- **Wrong overlap condition:** Using `s > lastEnd` instead of `s >= lastEnd`. If `s == lastEnd`, intervals are non-overlapping and both can be kept.
- **Counting removals instead of kept:** The problem asks for minimum removals. It's easier to compute maximum kept, then subtract from total.
- **Not handling empty input:** `n == 0` should return 0.

14. Edge Cases

Input	Expected	Notes
<code>[]</code>	0	Empty
<code>[[1,2]]</code>	0	Single interval
<code>[[1,2], [1,2], [1,2]]</code>	2	All identical
<code>[[1,5], [2,3], [3,4]]</code>	1	Keep <code>[2,3], [3,4]</code> or <code>[1,5]</code>
<code>[[1,2], [2,3], [3,4], [4,5]]</code>	0	All touching, no overlap
<code>[[1,100], [2,3], [4,5], [6,7]]</code>	1	Keep all small ones

15. Variations

15.1 Weighted Interval Scheduling

Each interval has a weight/profit. Maximize total profit of non-overlapping intervals. Requires DP ($O(n^2)$ or $O(n \log n)$ with binary search).

15.2 Maximum Length of Pair Chain

Similar but with pairs (a, b) where $a < b$. Chain $(a, b) \rightarrow (c, d)$ if $b < c$. Same greedy: sort by end.

15.3 N Meetings in One Room

Given start and end times, find maximum number of meetings that can be held in one room. Same greedy algorithm.

16. Related Algorithms/Data Structures

Algorithm	Connection	When to Choose
Merge Intervals	Merges overlaps instead of removing	When you want combined coverage
Sweep Line	Counts overlaps at each point	When you need overlap count, not removal
Weighted Interval Scheduling (DP)	Adds weights to intervals	When intervals have values/profits
Activity Selection	Same greedy, classic problem	Same as non-overlapping intervals

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
N meetings in one room	GFG	Classic greedy	Easy
Maximum Length of Pair Chain	LeetCode 646	Sort by end	Medium

Medium

Problem	Platform	Pattern	Difficulty
Non-overlapping Intervals	LeetCode 435	Min removals	Medium
Minimum Number of Arrows to Burst Balloons	LeetCode 452	Sort by end	Medium

Hard

Problem	Platform	Pattern	Difficulty
Maximum Number of Events That Can Be Attended II	LeetCode 1751	Weighted + DP	Hard

Problem	Platform	Pattern	Difficulty
Course Schedule III	LeetCode 630	Greedy + max-heap	Hard

18. Interview Explanation

"Non-overlapping Intervals is solved by the Earliest Finish Time greedy strategy. I sort intervals by end time, then iterate: keep the interval if its start is at or after the last kept interval's end; otherwise skip it. The minimum removals equals total intervals minus the maximum kept. This works because choosing the earliest-finishing interval always leaves the most room for remaining intervals. Time is $O(n \log n)$ due to sorting."

19. Revision Notes

- **Sort by end** — critical for the greedy to work.
- **Keep condition:** `start >= lastEnd`
- **Answer:** `n - kept`
- **Greedy proof:** Earliest finish time is optimal.
- **Complexity:** $O(n \log n)$ time, $O(1)$ space.
- **Trap:** Sorting by start gives wrong answer.

20. Final Cheat Sheet

Aspect	Details
When to use	Max non-overlapping intervals, min removals for non-overlap
Main operations	Sort by end, greedy keep
Time	$O(n \log n)$
Space	$O(1)$
Key code	<code>sort by end; if (s >= lastEnd) { kept++; lastEnd = e; }</code>
Edge cases	Empty, single, all touching, all overlapping
Common trap	Sorting by start, <code>></code> vs <code>>=</code> for overlap

MEETING ROOMS

1. Overview

Meeting Rooms (LeetCode 252) asks: Given an array of meeting time intervals, determine if a person can attend all meetings. That is, check if any two meetings overlap.

Example: `[[0,30], [5,10], [15,20]]` → `false` (0-30 overlaps with 5-10 and 15-20)

2. Intuition

If you have a list of meetings, you can attend all of them only if no two meetings overlap. This is the simplest interval problem.

Analogy: If you have a calendar and you check if any two events overlap, you sort by start time and check if any meeting starts before the previous one ends.

Step-by-step reasoning:

1. Sort intervals by start time.
2. For each adjacent pair, check if the first ends before the second starts.
3. If any pair overlaps, return false.

3. When to Use It

Situation	Example Problem Phrase
Check if a person can attend all meetings	"Can attend all meetings"
Check if any two intervals overlap	"Determine if there is any conflict"
Simple overlap detection in sorted intervals	"Check if intervals are non-overlapping"

Trigger phrases: "can attend all meetings", "no conflicts", "non-overlapping", "no overlap"

4. When Not to Use It

- **When you need to count overlaps, not just detect:** Use sweep line or Meeting Rooms II.
- **When you need to find the minimum number of rooms required:** Use Meeting Rooms II (min-heap or sweep line).
- **When you need to merge intervals:** Use Merge Intervals instead.
- **When intervals are already sorted:** The overlap check is $O(n)$ without sorting.

5. Core Concepts

5.1 Adjacent Overlap Check

After sorting by start, only adjacent intervals need to be checked. If `intervals[i][1] > intervals[i+1][0]`, there's an overlap.

5.2 Transitivity

If adjacent intervals are non-overlapping, all intervals are non-overlapping (since they're sorted by start).

6. Step-by-Step Algorithm

Input: `intervals[][]`

1. If `intervals` has ≤ 1 element, return true.
2. Sort intervals by start time.
3. For `i = 0` to `n-2`:
 - a. If `intervals[i][1] > intervals[i+1][0]`:

```
        return false // overlap found
4. Return true.
```

7. Dry Run

Input: `[[0,30], [5,10], [15,20]]`

Step 1: Sort by start (already sorted).

i	Intervals[i]	Intervals[i+1]	Overlap? (end > next start)
0	<code>[0,30]</code>	<code>[5,10]</code>	<code>30 > 5</code> ✓ → false

Answer: `false`

Input: `[[7,10], [2,4]]`

Step 1: Sort by start → `[[2,4], [7,10]]`

i	Intervals[i]	Intervals[i+1]	Overlap?
0	<code>[2,4]</code>	<code>[7,10]</code>	<code>4 > 7?</code> No X

Answer: `true`

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

bool canAttendMeetings(vector<vector<int>>& intervals) {
    int n = intervals.size();
    if (n <= 1) return true;

    // Sort by start time
    sort(intervals.begin(), intervals.end());

    for (int i = 0; i < n - 1; i++) {
        if (intervals[i][1] > intervals[i+1][0]) {
            return false; // Overlap found
        }
    }

    return true;
}

// Example usage
int main() {
    vector<vector<int>> intervals1 = {{0,30}, {5,10}, {15,20}};
    cout << boolalpha << canAttendMeetings(intervals1) << endl; // false
}
```

```
vector<vector<int>> intervals2 = {{7,10}, {2,4}};
cout << boolalpha << canAttendMeetings(intervals2) << endl; // true

return 0;
}
```

9. Python Implementation

```
def can_attend_meetings(intervals):
    n = len(intervals)
    if n <= 1:
        return True

    # Sort by start time
    intervals.sort(key=lambda x: x[0])

    for i in range(n - 1):
        if intervals[i][1] > intervals[i + 1][0]:
            return False

    return True

# Example usage
if __name__ == "__main__":
    print(can_attend_meetings([[0,30], [5,10], [15,20]])) # False
    print(can_attend_meetings([[7,10], [2,4]])) # True
```

10. Code Explanation

- **Sorting:** `sort(intervals.begin(), intervals.end())` sorts by start time.
- **Loop:** Check adjacent pairs. If `intervals[i][1] > intervals[i+1][0]`, meeting `i` ends after meeting `i+1` starts → overlap.
- **Return:** `true` if no overlaps found, `false` otherwise.

11. Complexity Analysis

Operation	Time	Space
Sorting	$O(n \log n)$	$O(\log n)$
Single pass	$O(n)$	$O(1)$
Overall	$O(n \log n)$	$O(\log n)$

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Simple overlap check	"Can attend all"	Sort + adjacent check	Meeting Rooms (LC 252)
Minimum rooms needed	"Minimum conference rooms"	Min-heap or sweep line	Meeting Rooms II (LC 253)
Maximum concurrent meetings	"Maximum simultaneous meetings"	Sweep line	My Calendar III

13. Common Mistakes

- **Wrong overlap condition:** Using $\geq$ instead of $>$. If one meeting ends exactly when another starts, you can attend both (if travel time is 0).
- **Not sorting:** Without sorting, checking adjacent pairs is meaningless.
- **Checking all pairs:** $O(n^2)$ approach is unnecessary. Sorting + $O(n)$ is sufficient.

14. Edge Cases

Input	Expected	Notes
<code>[]</code>	<code>true</code>	Empty
<code>[[1,5]]</code>	<code>true</code>	Single meeting
<code>[[1,5], [5,10]]</code>	<code>true</code>	Touching (end = start of next)
<code>[[1,5], [2,3]]</code>	<code>false</code>	Nested overlap
<code>[[1,5], [2,6]]</code>	<code>false</code>	Partial overlap

15. Variations

15.1 Meeting Rooms II (Minimum Rooms)

Find the minimum number of conference rooms needed. Use min-heap or sweep line.

15.2 Meeting Rooms III (Most Booked Room)

Find which room gets booked the most. More complex simulation.

16. Related Algorithms/Data Structures

Algorithm	Connection	When to Choose
Meeting Rooms II	Extends to count rooms	When you need minimum rooms
Sweep Line	Counts overlaps at each point	Alternative to min-heap
Merge Intervals	Merges overlaps	When you want combined coverage

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Meeting Rooms	LeetCode 252	Simple overlap check	Easy
Can Attend All Meetings	GFG	Same as above	Easy

Medium

Problem	Platform	Pattern	Difficulty
Meeting Rooms II	LeetCode 253	Min rooms	Medium
Minimum Platforms	GFG	Same as Meeting Rooms II	Medium

Hard

Problem	Platform	Pattern	Difficulty
Meeting Rooms III	LeetCode 2402	Most booked room	Hard

18. Interview Explanation

"Meeting Rooms is a simple overlap detection problem. I sort intervals by start time, then check adjacent pairs. If any meeting ends after the next one starts, they overlap and I return false. Otherwise, all meetings can be attended. Time is $O(n \log n)$ due to sorting."

19. Revision Notes

- **Sort by start** — needed for adjacent check.
- **Overlap condition:** `end > next_start`
- **Touching is allowed:** `end == next_start` is fine.
- **Complexity:** $O(n \log n)$ time, $O(1)$ space.

20. Final Cheat Sheet

Aspect	Details
When to use	Check if any two intervals overlap
Main operations	Sort by start, adjacent check
Time	$O(n \log n)$
Space	$O(1)$
Key code	<code>if (intervals[i][1] > intervals[i+1][0]) return false;</code>
Edge cases	Empty, single, touching at endpoint
Common trap	Using <code>>=</code> instead of <code>></code> for overlap

MINIMUM PLATFORMS

1. Overview

Minimum Platforms (also called Meeting Rooms II) asks: Given arrival and departure times of trains/meetings, find the minimum number of platforms/conference rooms required so that no train/meeting waits.

Example: Arrivals = [900, 940, 950, 1100, 1500, 1800], Departures = [910, 1200, 1120, 1130, 1900, 2000] → Minimum platforms = 3

2. Intuition

Think of a railway station. Trains arrive and depart. At any given time, multiple trains may be at the station. The maximum number of trains present simultaneously is the minimum number of platforms needed.

Analogy: Imagine a timeline. Every time a train arrives, the platform count increases by 1. Every time a train departs, the count decreases by 1. The peak of this count is the answer.

Step-by-step reasoning:

1. Separate arrivals and departures into two sorted lists.
2. Use two pointers: one for arrivals, one for departures.
3. Walk through time: if the next event is an arrival, increment count; if it's a departure, decrement count.
4. Track the maximum count.

Why this works: The maximum simultaneous count is the minimum number of platforms needed because each platform can serve one train at a time.

3. When to Use It

Situation	Example Problem Phrase
Minimum number of rooms/platforms needed	"Minimum number of conference rooms"
Maximum concurrent events	"Maximum number of simultaneous meetings"
Resource allocation for overlapping events	"Minimum platforms required"
Peak load calculation	"Maximum number of trains at station"

Trigger phrases: "minimum platforms", "minimum rooms", "maximum simultaneous", "minimum number of", "conference rooms"

4. When Not to Use It

- **When you only need to check if all meetings can be attended (no overlap):** Use Meeting Rooms (simple check).
- **When intervals are few and you need a simple answer:** Manual inspection might be faster.
- **When you need to merge intervals instead:** Use Merge Intervals.
- **When you need to simulate which specific rooms are used:** Use a min-heap approach (Meeting Rooms II variant).

5. Core Concepts

5.1 Event-Based Thinking

Instead of looking at intervals as blocks, think of them as two events: arrival (+1) and departure (-1).

5.2 Two-Pointer Technique

Sort both arrays and use two pointers to walk through events in chronological order.

5.3 Peak Concurrent Count

The answer is the maximum number of active intervals at any point in time.

6. Step-by-Step Algorithm

```
Input: arr[] (arrivals), dep[] (departures)

1. Sort arr[] and dep[].
2. Initialize i = 0, j = 0, count = 0, maxCount = 0.
3. While i < n and j < n:
  a. If arr[i] <= dep[j]:
    count++           // train arrives
    i++
  b. Else:
    count--           // train departs
    j++
  c. maxCount = max(maxCount, count)
4. Return maxCount.
```

7. Dry Run

Input: Arrivals = [900, 940, 950, 1100, 1500, 1800], Departures = [910, 1200, 1120, 1130, 1900, 2000]

Step 1: Sort both arrays (already sorted).

i	j	arr[i]	dep[j]	arr[i] ≤ dep[j]?	Count	maxCount	Action
0	0	900	910	Yes	1	1	Arrive 900
1	0	940	910	No	0	1	Depart 910
1	1	940	1200	Yes	1	1	Arrive 940
2	1	950	1200	Yes	2	2	Arrive 950
3	1	1100	1200	Yes	3	3	Arrive 1100
4	1	1500	1200	No	2	3	Depart 1200
4	2	1500	1120	No	1	3	Depart 1120

i	j	arr[i]	dep[j]	arr[i] ≤ dep[j]?	Count	maxCount	Action
4	3	1500	1130	No	0	3	Depart 1130
4	4	1500	1900	Yes	1	3	Arrive 1500
5	4	1800	1900	Yes	2	3	Arrive 1800
5	5	—	2000	—	1	3	Depart 1900
—	—	—	—	—	0	3	Depart 2000

Final Answer: 3

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

int findMinimumPlatforms(vector<int>& arr, vector<int>& dep) {
    int n = arr.size();
    if (n == 0) return 0;

    // Sort both arrays
    sort(arr.begin(), arr.end());
    sort(dep.begin(), dep.end());

    int i = 0, j = 0;
    int count = 0, maxCount = 0;

    while (i < n && j < n) {
        if (arr[i] <= dep[j]) {
            count++;
            i++;
        } else {
            count--;
            j++;
        }
        maxCount = max(maxCount, count);
    }

    return maxCount;
}

// Example usage
int main() {
    vector<int> arr = {900, 940, 950, 1100, 1500, 1800};
    vector<int> dep = {910, 1200, 1120, 1130, 1900, 2000};

    cout << "Minimum platforms: " << findMinimumPlatforms(arr, dep) << endl;
    // Output: 3
}
```

```
    return 0;  
}
```

10. Code Explanation

- **Sorting:** Both arrays are sorted independently. This lets us process events in chronological order.
- **Two pointers:** `i` tracks arrivals, `j` tracks departures.
- **Comparison:** `arr[i] <= dep[j]` — if the next arrival is at or before the next departure, we process the arrival (`count++`). Otherwise, we process the departure (`count--`).
- **Note on tie-breaking:** When `arr[i] == dep[j]`, we process the arrival first (using `<=`). This is important because at exactly the same time, a train arriving needs a platform before one departing frees one. However, some problems treat this differently (e.g., if the station can handle both simultaneously). Adjust based on problem constraints.

11. Complexity Analysis

Operation	Time	Space
Sorting	$O(n \log n)$	$O(\log n)$
Two-pointer pass	$O(n)$	$O(1)$
Overall	$O(n \log n)$	$O(\log n)$

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Minimum rooms/platforms	"Minimum number of rooms"	Sort + two-pointer	Meeting Rooms II (LC 253), Min Platforms (GFG)
Maximum overlap	"Maximum simultaneous"	Sweep line (events)	My Calendar III (LC 732)
Resource allocation	"Minimum resources needed"	Two-pointer or min-heap	Car Pooling (LC 1094)

13. Common Mistakes

- **Not sorting both arrays:** Both arrivals and departures must be sorted independently.
- **Wrong tie-breaking:** Whether `arr[i] <= dep[j]` or `arr[i] < dep[j]` matters. Usually `<=` is correct (arrival before departure at same time), but check the problem.
- **Forgetting to track max:** The count fluctuates; we need the maximum, not the final value.
- **Using intervals instead of separate arrays:** Converting to events (+1/-1) and sorting all events is an alternative (sweep line) approach.

14. Edge Cases

Input	Expected	Notes
<code>arr=[], dep=[]</code>	0	Empty
<code>arr=[1], dep=[2]</code>	1	Single train
<code>arr=[1,2], dep=[2,3]</code>	1	Touching (1-2, 2-3) — if arrival = departure, 2 platforms might be needed
<code>arr=[1,1,1], dep=[2,2,2]</code>	3	All arrive and depart together
<code>arr=[1,3,5], dep=[2,4,6]</code>	1	No overlap
<code>arr=[1,2,3], dep=[2,3,4]</code>	2	Overlapping chain

15. Variations

15.1 Min-Heap Approach (Meeting Rooms II)

Instead of two-pointer, use a min-heap to track end times. For each meeting sorted by start, remove ended meetings from heap, then add current. Max heap size = answer.

15.2 Sweep Line (Event-Based)

Create events `(time, +1)` for arrival and `(time, -1)` for departure. Sort all events, process in order, track max.

15.3 Car Pooling

Track passengers on a car. Trips have `(passengers, from, to)`. Use sweep line to ensure capacity is never exceeded.

16. Related Algorithms/Data Structures

Algorithm	Connection	When to Choose
Sweep Line	Same event-based approach	More general; handles multiple event types
Min-Heap	Tracks active end times	When you need to know which rooms are free
Difference Array	Range addition concept	Similar but for adding values to ranges

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Meeting Rooms	LeetCode 252	Simple overlap check	Easy

Problem	Platform	Pattern	Difficulty
Minimum Platforms	GFG	Two-pointer	Medium

Medium

Problem	Platform	Pattern	Difficulty
Meeting Rooms II	LeetCode 253	Min-heap / two-pointer	Medium
Car Pooling	LeetCode 1094	Sweep line	Medium
Corporate Flight Bookings	LeetCode 1109	Difference array	Medium

Hard

Problem	Platform	Pattern	Difficulty
My Calendar III	LeetCode 732	Sweep line / segment tree	Hard
Maximum Number of Events That Can Be Attended	LeetCode 1353	Greedy + min-heap	Hard

18. Interview Explanation

"Minimum Platforms is solved by treating arrivals and departures as events. I sort both arrays, then use two pointers: when a train arrives I increment count, when it departs I decrement. The maximum count seen is the answer. This works because the peak number of simultaneous trains equals the minimum platforms needed. Time is $O(n \log n)$ for sorting."

19. Revision Notes

- **Sort both arrays** — arrivals and departures independently.
- **Two-pointer:** `arr[i] <= dep[j]` → arrive; else → depart.
- **Track max count** — answer is max count, not final.
- **Tie-breaking:** Arrival before departure at same time (usually).
- **Complexity:** $O(n \log n)$ time, $O(1)$ space.

20. Final Cheat Sheet

Aspect	Details
When to use	Minimum rooms/platforms, max concurrent events
Main operations	Sort, two-pointer
Time	$O(n \log n)$
Space	$O(1)$
Key code	<code>if (arr[i] <= dep[j]) count++; else count--;</code>

Aspect	Details
Edge cases	Empty, single, all same time, touching
Common trap	Wrong tie-breaking, not tracking max

SWEEP LINE

1. Overview

Sweep Line is a geometric/algorithmic paradigm where an imaginary vertical line sweeps across the plane from left to right, processing events as it encounters them. In the context of intervals, it converts interval problems into event-processing problems.

At its core, sweep line processes "events" (start and end points) in sorted order, maintaining a running state that gets updated at each event.

2. Intuition

Imagine a vertical line moving from left to right across a timeline. When the line hits the start of an interval, that interval becomes "active". When it hits the end, the interval becomes "inactive". By tracking the number of active intervals at any point, we can answer many questions.

Analogy: Think of a busy street. You stand at a point and count how many ongoing events (parades, marathons, etc.) are currently active. As you walk from left to right, you increment the count when you see the start of an event and decrement when you see the end.

Step-by-step reasoning:

1. For each interval, create two events: (start, +1) and (end, -1).
2. Sort all events by time.
3. Process events in order, updating a running count.
4. The count at any time represents the number of active intervals.

3. When to Use It

Situation	Example Problem Phrase
Find maximum overlap count	"Maximum number of simultaneous meetings"
Find if any point is covered by more than K intervals	"Is there a point with triple booking?"
Compute coverage at each point	"How many intervals cover each point?"
Problems involving multiple event types	"Start and end of each event"
Insert/delete intervals dynamically	"Add a range, query a point"

Trigger phrases: "sweep line", "maximum overlap", "simultaneous", "concurrent", "event processing", "calendar booking", "number of intervals covering"

4. When Not to Use It

- **When you only need to merge intervals:** Merge Intervals is simpler ($O(n \log n)$ vs $O(n \log n)$ but simpler code).
- **When you need range queries/updates (not point queries):** Use segment tree or Fenwick tree.
- **When intervals are small and static:** Simple nested loops might be sufficient.
- **When you need to query many arbitrary points:** Use a prefix sum or difference array after building.
- **When the number of events is very large (e.g., 10^9):** Coordinate compress first.

5. Core Concepts

5.1 Events

Each interval produces two events: $(\text{start_time}, +1)$ and $(\text{end_time}, -1)$. The $+1$ means "start covering", -1 means "stop covering".

5.2 Sorting

Events are sorted by time. If multiple events have the same time, the order matters (start events before end events, or vice versa, depending on the problem).

5.3 Running Count

As we process events, we maintain a running count of active intervals. This is the core state.

5.4 Maximum/Peeks

For many problems, we track the maximum value of the running count (peak overlap).

6. Step-by-Step Algorithm

General Sweep Line for Overlap Problems:

1. Create events list: for each interval $[s, e]$:
 - Add $(s, +1)$ // start
 - Add $(e, -1)$ // end
2. Sort events by time.
 - If times are equal, process $+1$ before -1 (or vice versa based on problem).
3. Initialize $\text{count} = 0$, $\text{maxCount} = 0$ (or other state).
4. For each event $(\text{time}, \text{delta})$:
 - a. $\text{count} += \text{delta}$
 - b. Update $\text{maxCount} = \max(\text{maxCount}, \text{count})$ [if needed]
 - c. Check conditions (e.g., if $\text{count} > K$, return false)
5. Return result $(\text{maxCount}, \text{true/false}, \text{etc.})$.

7. Dry Run

Input: `Intervals = [[1,4], [2,5], [7,9]]` — Find maximum overlap.

Events:

Event	Time	Delta
Start	1	+1
Start	2	+1
End	4	-1
End	5	-1
Start	7	+1
End	9	-1

Processing (sorted by time):

Time	Delta	Count	MaxCount
1	+1	1	1
2	+1	2	2
4	-1	1	2
5	-1	0	2
7	+1	1	2
9	-1	0	2

Maximum overlap: 2 (between times 2 and 4)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct Event {
    int time;
    int delta; // +1 for start, -1 for end
};

int maxOverlap(vector<vector<int>>& intervals) {
    if (intervals.empty()) return 0;

    vector<Event> events;
    for (auto& iv : intervals) {
        events.push_back({iv[0], 1});
        events.push_back({iv[1], -1});
    }
}
```

```

    }

    // Sort by time; if time is same, process start (+1) before end (-1)
    // because at the exact same time, we count the new interval as active
    sort(events.begin(), events.end(),
        [](const Event& a, const Event& b) {
            if (a.time != b.time) return a.time < b.time;
            return a.delta > b.delta; // +1 before -1
        });

    int count = 0, maxCount = 0;
    for (auto& e : events) {
        count += e.delta;
        maxCount = max(maxCount, count);
    }

    return maxCount;
}

// Example usage
int main() {
    vector<vector<int>> intervals = {{1,4}, {2,5}, {7,9}};
    cout << "Maximum overlap: " << maxOverlap(intervals) << endl; // 2

    return 0;
}

```

9. Python Implementation

```

def max_overlap(intervals):
    if not intervals:
        return 0

    events = []
    for s, e in intervals:
        events.append((s, 1)) # start
        events.append((e, -1)) # end

    # Sort by time; +1 before -1 for same time
    events.sort(key=lambda x: (x[0], -x[1]))

    count = 0
    max_count = 0

    for time, delta in events:
        count += delta
        max_count = max(max_count, count)

    return max_count

```

```
# Example usage
if __name__ == "__main__":
    print(max_overlap([[1,4], [2,5], [7,9]])) # 2
```

10. Code Explanation

- **Event struct:** Stores **time** and **delta** (+1 for start, -1 for end).
- **Building events:** For each interval, create two events.
- **Sorting:** Sort by time. For equal times, **+1** (start) comes before **-1** (end) because at the boundary, the interval is considered active. This is controlled by **a.delta > b.delta**.
- **Processing:** For each event, add delta to count. Track max.
- **Return:** The maximum overlap count.

11. Complexity Analysis

Operation	Time	Space
Building events	O(n)	O(n)
Sorting	O(n log n)	O(log n)
Processing	O(n)	O(1)
Overall	O(n log n)	O(n)

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Max overlap count	"Maximum simultaneous"	Basic sweep line	My Calendar III
Check if triple booking	"No triple booking"	Sweep line, check count > 2	My Calendar II
Car pooling	"Capacity check"	Sweep line with capacity	Car Pooling
Employee free time	"Free time"	Sweep line, find gaps	Employee Free Time
Skyline problem	"Skyline"	Sweep line with multiset	The Skyline Problem

13. Common Mistakes

- **Wrong tie-breaking:** When two events have the same time, should start (+1) come before end (-1) or after? It depends on the problem. For "maximum overlap", start before end is usually correct. For "no triple booking", it depends on whether a booking at the exact boundary counts.
- **Not coordinate compressing:** For large time ranges (10^9), sorting events is fine, but if you need array operations, compress.

- **Forgetting about multiple events at the same time:** Processing them one by one without considering all simultaneous events can lead to wrong intermediate max.
- **Integer overflow:** If using large time values, use `long long`.

14. Edge Cases

Input	Expected	Notes
<code>[]</code>	0	Empty
<code>[[1,2]]</code>	1	Single interval
<code>[[1,5], [1,5]]</code>	2	Identical
<code>[[1,2], [2,3]]</code>	1 or 2	Depends on tie-breaking rule
<code>[[1,10], [2,3], [4,5]]</code>	2	Nested
<code>[[1,2], [3,4], [5,6]]</code>	1	No overlap

15. Variations

15.1 Sweep Line with Multiset

For problems like The Skyline Problem, we need to track the maximum height among active intervals, not just the count. Use a multiset (max-heap with lazy deletion).

15.2 Sweep Line with Coordinate Compression

When time values are large but sparse, compress them to indices for array-based operations.

15.3 Sweep Line with Segment Tree

For complex queries during sweep (e.g., range queries on active intervals), combine with segment tree.

16. Related Algorithms/Data Structures

Algorithm	Connection	When to Choose
Difference Array	Same +1/-1 concept but on an array	When range is small and you need array queries
Segment Tree	Range queries with updates	When you need complex queries during sweep
Min-Heap	Tracks active end times	Alternative to sweep line for min rooms
Interval Tree	Dynamic overlap queries	When intervals are inserted/deleted

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Meeting Rooms	LeetCode 252	Simple overlap check	Easy
Minimum Platforms	GFG	Two-pointer	Medium

Medium

Problem	Platform	Pattern	Difficulty
My Calendar II	LeetCode 731	Sweep line, check double booking	Medium
Car Pooling	LeetCode 1094	Sweep line, capacity check	Medium
Corporate Flight Bookings	LeetCode 1109	Difference array	Medium

Hard

Problem	Platform	Pattern	Difficulty
My Calendar III	LeetCode 732	Sweep line, max overlap	Hard
The Skyline Problem	LeetCode 218	Sweep line with multiset	Hard
Employee Free Time	LeetCode 759	Sweep line, find gaps	Hard

18. Interview Explanation

"Sweep Line is a general technique for interval problems. I convert each interval into two events — a start (+1) and an end (-1) — then sort all events by time. Processing them in order gives me a running count of active intervals. The maximum of this count is the peak overlap. The key detail is tie-breaking: when start and end events have the same time, I typically process starts first. This gives $O(n \log n)$ time due to sorting."

19. Revision Notes

- **Events:** Each interval $\rightarrow (\text{start}, +1), (\text{end}, -1)$.
- **Sort by time;** tie-break: $+1$ before -1 (usually).
- **Running count:** Add delta, track max.
- **Use cases:** Max overlap, check booking limits, capacity checks.
- **Complexity:** $O(n \log n)$ time, $O(n)$ space.

20. Final Cheat Sheet

Aspect	Details
When to use	Overlap counting, max concurrent, booking limits
Main operations	Events creation, sorting, sweep
Time	$O(n \log n)$

Aspect	Details
Space	$O(n)$
Key code	<pre>events.push_back({time, delta}); sort(events); for (e: events) count += e.delta;</pre>
Edge cases	Empty, single, same time events
Common trap	Tie-breaking order, not tracking max

DIFFERENCE ARRAY EVENTS

1. Overview

Difference Array is a technique for efficiently applying multiple range additions (add a value to every element in a range) and then querying the final value at each point. Instead of updating each element in the range individually ($O(n)$ per update), we use $O(1)$ per update by marking the start and end of each range, then compute the final array with a prefix sum.

Example: Add 10 to range [2, 5], add 20 to range [3, 6] → Final array at indices 0..7: [0, 0, 10, 30, 30, 30, 20, 0]

2. Intuition

Think of a row of buckets. Instead of pouring water into every bucket in a range one by one, you mark at the start "start pouring here" and at the end+1 "stop pouring here". Then you walk once from left to right, pouring as you go.

Analogy: Imagine a long hallway with lights. Instead of turning on each light in a range individually, you place a switch at the start that turns them on and a switch at the end+1 that turns them off. Then you walk through once, flipping switches as you go.

Step-by-step reasoning:

1. Create an array `diff` of size $n+1$, initialized to 0.
2. For each range update `[l, r]` with value `val`:
 - `diff[l] += val`
 - `diff[r+1] -= val`
3. Compute prefix sum: `arr[i] = arr[i-1] + diff[i]` (or `arr[i] = diff[i] + arr[i-1]`).

Why this works: The difference array captures changes at boundaries. The prefix sum reconstructs the actual values.

3. When to Use It

Situation	Example Problem Phrase
Range add operations followed by point queries	"Add K to range [L,R] for all queries"

Situation	Example Problem Phrase
Multiple range updates, query final array	"After all operations, find the value at each index"
Counting how many intervals cover each point	"Number of intervals covering each point"
Sweep line on a fixed range	"Find the maximum value after all range updates"

Trigger phrases: "range addition", "range update", "add to range", "difference array", "prefix sum after range updates", "multiple range operations"

4. When Not to Use It

- **When you need to query intermediate states (after each update):** Difference array only gives the final state efficiently. Use a segment tree or Fenwick tree for online queries.
- **When the range is very large (10^9) without compression:** Use a map-based difference array (coordinate compress) or use a segment tree with lazy propagation.
- **When you need range queries, not point queries:** After building the final array, you can compute prefix sums for range queries, but if updates and queries are interleaved, use a segment tree.
- **When you need to merge intervals (not count coverage):** Use Merge Intervals.

5. Core Concepts

5.1 Difference Array

An array `diff` where `diff[i]` stores the net change at index `i`. Adding `val` to `[l, r]` means `diff[l] += val` and `diff[r+1] -= val`.

5.2 Prefix Sum

After processing all updates, the actual value at index `i` is the prefix sum of `diff[0..i]`.

5.3 Boundary Convention

The update `[l, r]` is inclusive. `diff[r+1]` is used to stop the effect after `r`.

6. Step-by-Step Algorithm

Input: `n` (size of array), updates `[(l, r, val), ...]`

1. Initialize `diff` array of size `n+1` with 0.
2. For each update `(l, r, val)`:
 - a. `diff[l] += val`
 - b. if `r+1 < n`: `diff[r+1] -= val`
3. Initialize `arr` of size `n`.
4. `arr[0] = diff[0]`
5. For `i = 1` to `n-1`:
 - a. `arr[i] = arr[i-1] + diff[i]` // prefix sum
6. Return `arr`.

7. Dry Run

Input: $n = 8$, Updates: $[2, 5, 10]$, $[3, 6, 20]$

Step 1: Initialize $\text{diff} = [0, 0, 0, 0, 0, 0, 0, 0, 0]$ (size $n+1$).

Step 2: Apply updates.

Update	l	r	val	$\text{diff}[l] += \text{val}$	$\text{diff}[r+1] -= \text{val}$
1	2	5	10	$\text{diff}[2] = 10$	$\text{diff}[6] = -10$
2	3	6	20	$\text{diff}[3] = 20$	$\text{diff}[7] = -20$

After updates: $\text{diff} = [0, 0, 10, 20, 0, 0, -10, -20, 0]$

Step 3: Prefix sum.

i	$\text{diff}[i]$	$\text{arr}[i] = \text{arr}[i-1] + \text{diff}[i]$	Value
0	0	0	0
1	0	$0 + 0 = 0$	0
2	10	$0 + 10 = 10$	10
3	20	$10 + 20 = 30$	30
4	0	$30 + 0 = 30$	30
5	0	$30 + 0 = 30$	30
6	-10	$30 + (-10) = 20$	20
7	-20	$20 + (-20) = 0$	0

Final Answer: $\text{arr} = [0, 0, 10, 30, 30, 30, 20, 0]$

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

vector<int> applyRangeUpdates(int n, vector<vector<int>>& updates) {
    // diff array of size n+1
    vector<int> diff(n + 1, 0);

    // Apply each range update
    for (auto& upd : updates) {
        int l = upd[0], r = upd[1], val = upd[2];
        diff[l] += val;
        if (r + 1 < n) {
            diff[r + 1] -= val;
        }
    }
}
```

```

    // Build final array using prefix sum
    vector<int> arr(n);
    arr[0] = diff[0];
    for (int i = 1; i < n; i++) {
        arr[i] = arr[i - 1] + diff[i];
    }

    return arr;
}

// Example usage: count how many intervals cover each point
vector<int> intervalCoverage(int n, vector<vector<int>>& intervals) {
    // Each interval [l, r] adds 1 to coverage
    vector<vector<int>> updates;
    for (auto& iv : intervals) {
        updates.push_back({iv[0], iv[1], 1});
    }
    return applyRangeUpdates(n, updates);
}

int main() {
    vector<vector<int>> updates = {{2, 5, 10}, {3, 6, 20}};
    auto result = applyRangeUpdates(8, updates);

    cout << "Final array: ";
    for (int x : result) cout << x << " ";
    // Output: 0 0 10 30 30 30 20 0

    return 0;
}

```

9. Python Implementation

```

def apply_range_updates(n, updates):
    # diff array of size n+1
    diff = [0] * (n + 1)

    # Apply each range update
    for l, r, val in updates:
        diff[l] += val
        if r + 1 < n:
            diff[r + 1] -= val

    # Build final array using prefix sum
    arr = [0] * n
    arr[0] = diff[0]
    for i in range(1, n):
        arr[i] = arr[i - 1] + diff[i]

    return arr

```

```
def interval_coverage(n, intervals):
    """Count how many intervals cover each point."""
    updates = [[l, r, 1] for l, r in intervals]
    return apply_range_updates(n, updates)

# Example usage
if __name__ == "__main__":
    updates = [[2, 5, 10], [3, 6, 20]]
    result = apply_range_updates(8, updates)
    print(f"Final array: {result}")
    # Output: [0, 0, 10, 30, 30, 30, 20, 0]
```

10. Code Explanation

- **Diff array:** Size $n+1$ to safely handle $r+1$ for $r = n-1$.
- **Update application:** `diff[l] += val` — start adding at l . `diff[r+1] -= val` — stop adding after r .
- **Prefix sum:** `arr[i] = arr[i-1] + diff[i]` — reconstructs the actual values.
- **Boundary check:** `if (r + 1 < n)` prevents out-of-bounds access. If $r+1 == n$, we can skip because `diff[n]` is never used in the prefix sum.

11. Complexity Analysis

Operation	Time	Space
Building diff array	$O(n)$	$O(n)$
Each update	$O(1)$	$O(1)$
Prefix sum pass	$O(n)$	$O(1)$
Overall	$O(n + m)$ where m = number of updates	$O(n)$

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Range addition, final array	"Add K to range for all queries"	Diff array + prefix sum	Range Addition (LC 370)
Interval coverage count	"Number of intervals covering each point"	Diff array with $+1/-1$	Corporate Flight Bookings (LC 1109)
Maximum after range adds	"Maximum value after all operations"	Diff array, track max during prefix sum	Range Addition (LC 370)
Sweep line on array	"Count overlaps on discrete points"	Diff array (discrete version of sweep line)	Car Pooling (LC 1094)

13. Common Mistakes

- **Off-by-one on $r+1$:** Forgetting to subtract at $r+1$ is the most common mistake. Every range addition must have a corresponding end marker.
- **Array size:** `diff` must be $n+1$ to safely handle $r+1 = n$.
- **Inclusive vs exclusive ranges:** The standard difference array works on inclusive ranges $[l, r]$. If the problem uses exclusive ranges, adjust accordingly.
- **Not handling updates to the same index:** If multiple updates apply to the same l or $r+1$, values accumulate correctly.
- **Confusing with prefix sum:** Difference array is the inverse of prefix sum. Don't confuse the two.

14. Edge Cases

Input	Expected	Notes
$n=0$	<code>[]</code>	Empty array
No updates	All zeros	No changes
Single update full range	<code>[val, val, ...]</code>	Update $[0, n-1]$
Overlapping updates	Sum of values	Correct accumulation
Updates at exact boundaries	Correct at edges	$r+1$ may be out of bounds

15. Variations

15.1 Map-Based Difference Array (Sparse)

When the range is large but updates are sparse, use a `map<int, int>` instead of an array. Each update adds to `diff[l]` and subtracts from `diff[r+1]`. The prefix sum is computed by iterating the map in sorted order.

15.2 2D Difference Array

For 2D range updates. `diff[x1][y1] += val, diff[x1][y2+1] -= val, diff[x2+1][y1] -= val, diff[x2+1][y2+1] += val`. Then 2D prefix sum.

15.3 Difference Array with Point Queries

If you need to query a point after some updates but before all, use a Fenwick tree (BIT) for online point queries.

16. Related Algorithms/Data Structures

Algorithm	Connection	When to Choose
Prefix Sum	Inverse of difference array	When you need to query range sums, not apply updates
Fenwick Tree (BIT)	Supports point updates and range queries	When updates and queries are interleaved
Segment Tree	Range updates and range queries	When you need both range updates and range queries

Algorithm	Connection	When to Choose
Sweep Line	Same +1/-1 concept for events	When intervals are not on a fixed discrete array

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Range Addition	LeetCode 370	Basic diff array	Medium
Corporate Flight Bookings	LeetCode 1109	Diff array	Medium

Medium

Problem	Platform	Pattern	Difficulty
Car Pooling	LeetCode 1094	Diff array on timeline	Medium
Minimum Number of Operations to Move All Balls to Each Box	LeetCode 1769	Diff array + prefix	Medium
Shifting Letters II	LeetCode 2381	Diff array on characters	Medium

Hard

Problem	Platform	Pattern	Difficulty
Maximum Number of Events That Can Be Attended	LeetCode 1353	Diff array + greedy	Hard
Stamping the Sequence	LeetCode 936	Diff array + reverse simulation	Hard

18. Interview Explanation

"Difference Array is a technique for efficiently applying multiple range additions. Instead of updating each element in a range, I maintain a diff array where `diff[l] += val` marks the start of addition and `diff[r+1] -= val` marks the end. After processing all updates, a single prefix sum pass reconstructs the final array. Each update is $O(1)$ and the final reconstruction is $O(n)$. This is ideal for problems with many range updates followed by a single query."

19. Revision Notes

- **Key ops:** `diff[l] += val; diff[r+1] -= val;`
- **Final array:** Prefix sum of diff.
- **Size:** `diff` of size `n+1`.

- **Each update:** $O(1)$.
- **Reconstruction:** $O(n)$.
- **Use for:** Range add $\rightarrow$ final values, interval coverage count.
- **Sparse variant:** Use `map<int, int>` for large ranges.

20. Final Cheat Sheet

Aspect	Details
When to use	Range additions, then query final values
Main operations	<code>diff[l] += val; diff[r+1] -= val;</code>
Time	$O(n + m)$ for m updates
Space	$O(n)$
Key code	<code>diff[l] += val; if (r+1 < n) diff[r+1] -= val;</code>
Edge cases	$r = n-1$, empty updates, overlapping updates
Common trap	Forgetting <code>r+1</code> decrement, off-by-one on array size

LINE SWEEP WITH SORTING

1. Overview

Line Sweep with Sorting is a specific application of the sweep line paradigm where we sort all events (interval starts and ends) and then process them in order. This is the most common form of sweep line used in interval problems. The key distinction from basic sweep line is the explicit focus on sorting as the core operation that enables the $O(n \log n)$ solution.

2. Intuition

The fundamental insight: after sorting intervals by their start time, we can process them in a single pass, making decisions based on the current state. This is the workhorse of interval problems.

Analogy: Imagine organizing a busy day. You sort all your appointments by start time. Then you go through them one by one, keeping track of what's currently happening. This simple structure lets you answer many questions.

3. When to Use It

Situation	Example Problem Phrase
Any interval problem with overlaps	See all interval patterns
Problems that need chronological processing	"Process in order of time"
Problems where sorting simplifies the problem	"Sort intervals" is a hint
Problems with two or more event types	"Start and end times"

Trigger phrases: Any interval problem — the first step is almost always sorting.

4. When Not to Use It

- **When the array is tiny:** $O(n^2)$ might be simpler to code.
- **When intervals are already sorted and you need $O(n)$:** Don't re-sort.
- **When you need to process intervals in a different order:** Some problems process by end time (greedy), not start time.

5. Core Concepts

5.1 Sorting as Preprocessing

Sorting transforms an unordered set of intervals into a sequence we can process linearly.

5.2 Two-Pointer Sweep

For two arrays (arrivals, departures), use two independent pointers.

5.3 Event-Based Sweep

For intervals with $+1/-1$ events, sort all events together.

5.4 Greedy Sweep

After sorting by end time, greedily select intervals (Non-overlapping Intervals pattern).

6. Step-by-Step Algorithm

The algorithm varies by problem, but the core pattern is:

1. Sort intervals by some key (start, end, etc.).
2. Initialize state (count, lastEnd, result, etc.).
3. Process intervals in sorted order.
4. Return result.

7. Dry Run

See the dry runs under Merge Intervals, Non-overlapping Intervals, Minimum Platforms, and Sweep Line for specific examples.

8. C++ Implementation

The implementation depends on the specific problem. See the implementations under the individual algorithms for C++ code.

9. Python Implementation

See the individual algorithms for Python code.

10. Code Explanation

The key idea is always: sort first, then process in one pass. The specific logic during the pass depends on the problem.

11. Complexity Analysis

Operation	Time	Space
Sorting	$O(n \log n)$	$O(\log n)$
Single pass	$O(n)$	$O(1)$
Overall	$O(n \log n)$	$O(\log n)$

12. Common Patterns

Pattern	Sort By	Approach
Merge intervals	Start time	Merge overlapping
Max non-overlapping	End time	Greedy keep
Min platforms	Both (two-pointer)	Count +1/-1
Check overlap	Start time	Adjacent check
Insert interval	Already sorted	Three-phase

13. Common Mistakes

- **Sorting by the wrong key:** Sorting by start vs end gives different results. The greedy for non-overlapping intervals requires sorting by end.
- **Not sorting at all:** Many interval problems seem hard until you sort.
- **Modifying input inadvertently:** Sorting mutates the input array.

14. Edge Cases

See the edge cases under each individual algorithm.

15. Variations

15.1 Sort by Start

Used for merge intervals, meeting rooms, insert interval.

15.2 Sort by End

Used for non-overlapping intervals, minimum arrows to burst balloons.

15.3 Sort Both Arrays

Used for minimum platforms (two-pointer approach).

15.4 Sort Events with +1/-1

Used for general sweep line, calendar booking.

16. Related Algorithms/Data Structures

Algorithm	Connection	When to Choose
Sweep Line	The general paradigm	For complex event processing
Greedy	Sort + greedy is a common combo	For optimization problems
Two Pointers	Using two sorted arrays	For min platforms variant

17. Practice Problems

See the practice problems under each individual algorithm.

18. Interview Explanation

"Line sweep with sorting is the core technique for interval problems. The key insight is that sorting intervals by some criterion (start time, end time) transforms the problem into a linear scan. For example, sorting by start time lets us merge overlapping intervals in one pass; sorting by end time lets us greedily select non-overlapping intervals. The sorting step costs $O(n \log n)$, and the subsequent scan is $O(n)$."

19. Revision Notes

- **Sort first** — always the first step.
- **Choose the right sort key** — start for merge, end for greedy.
- **One-pass processing** — after sorting, $O(n)$.
- **Complexity:** $O(n \log n)$.

20. Final Cheat Sheet

Aspect	Details
When to use	Any interval problem
Main operations	Sort + one-pass processing
Time	$O(n \log n)$
Space	$O(\log n)$ for sorting
Key idea	Sorting transforms the problem
Common trap	Wrong sort key

CALENDAR BOOKING

1. Overview

Calendar Booking problems involve adding events to a calendar while ensuring no double/triple booking occurs. The three main variants are:

- **My Calendar I:** No double booking allowed (no overlap).
- **My Calendar II:** Double booking allowed, but no triple booking.
- **My Calendar III:** Return the maximum number of concurrent bookings (k-booking).

2. Intuition

Calendar I: Think of a physical calendar. You can only add an event if it doesn't overlap with any existing event. This is just an overlap check.

Calendar II: You can have overlapping events, but no point can have 3+ events. This is like a hotel where you can book two rooms but not three at the same time.

Calendar III: What's the peak occupancy? This is just the maximum overlap count.

Step-by-step reasoning:

- **Calendar I:** Store all intervals. For each new booking, check if it overlaps with any existing interval. If yes, reject.
- **Calendar II:** Track both "booked" intervals and "double-booked" intervals. When a new interval overlaps with a booked one, the overlap region becomes double-booked. If the new interval overlaps with any double-booked region, reject.
- **Calendar III:** Use sweep line. Maintain a running count of active intervals. The maximum count is the answer.

3. When to Use It

Situation	Example Problem Phrase
No overlapping events allowed	"No double booking"
Limit on concurrent events	"No triple booking"
Maximum concurrent events	"Maximum number of concurrent events"
Calendar/booking system	"My Calendar", "Book a meeting"

Trigger phrases: "calendar", "booking", "double booking", "triple booking", "my calendar"

4. When Not to Use It

- **When you just need to merge intervals:** Use Merge Intervals.
- **When you need to schedule the maximum number of events:** Use Non-overlapping Intervals (greedy).
- **When you need minimum rooms:** Use Meeting Rooms II.
- **When you need to insert many events without overlap checking:** Use interval tree.

5. Core Concepts

5.1 Calendar I — Overlap Check

For each new interval $[s, e]$, check if $s < \text{existing_end} \ \&\& \ e > \text{existing_start}$ for any existing interval. If found, reject.

5.2 Calendar II — Double Booking Tracking

Maintain two lists: `bookings` and `doubleBookings`. For a new booking, check overlap with `doubleBookings`. If none, add overlap regions with `bookings` to `doubleBookings`, then add the booking.

5.3 Calendar III — Sweep Line

Use events $(\text{start}, +1)$ and $(\text{end}, -1)$. Sort and process. Track max count.

6. Step-by-Step Algorithm

Calendar I (No Double Booking)

1. Store list of booked intervals.
2. For each new booking $[s, e]$:
 - a. For each existing $[b_s, b_e]$:
 - if $s < b_e$ and $e > b_s$: return false
 - b. Add $[s, e]$ to list.
 - c. Return true.

Calendar II (No Triple Booking)

1. Store booked list and double_booked list.
2. For new booking $[s, e]$:
 - a. For each pair in `double_booked`:
 - if $s < db_e$ and $e > db_s$: return false
 - b. For each pair in `booked`:
 - if $s < b_e$ and $e > b_s$:
 - overlap = $[\max(s, b_s), \min(e, b_e)]$
 - add overlap to `double_booked`
 - c. Add $[s, e]$ to `booked`.
 - d. Return true.

Calendar III (Max K-Booking)

1. Maintain events map (time -> delta).
2. For new booking $[s, e]$:
 - a. `events[s] += 1`
 - b. `events[e] -= 1`
 - c. Iterate sorted events in order, maintain running count.
 - d. Return max count.

7. Dry Run

Calendar II — Dry Run

Bookings: [10, 20], [15, 25], [20, 30]

Step 1: Book [10, 20]

- Double booked: empty → no conflict.
- `booked = [[10, 20]], double_booked = []`
- Return true.

Step 2: Book [15, 25]

- Check `double_booked`: empty → no conflict.
- Overlap with [10, 20]: `[max(15,10), min(25,20)] = [15, 20]`
- Add to `double_booked`.
- `booked = [[10, 20], [15, 25]], double_booked = [[15, 20]]`
- Return true.

Step 3: Book [20, 30]

- Check `double_booked`: [15, 20] — overlap? `20 < 20`? No (`20 == 20`, not `<`). No overlap.
- Overlap with [10, 20]: `[max(20,10), min(30,20)] = [20, 20]` — empty range, no overlap.
- Overlap with [15, 25]: `[max(20,15), min(30,25)] = [20, 25]` — add to `double_booked`.
- `booked = [[10, 20], [15, 25], [20, 30]]`
- `double_booked = [[15, 20], [20, 25]]`
- Return true.

Step 4: Book [10, 30]

- Check `double_booked`: [15, 20] — `10 < 20` and `30 > 15` → conflict! Return false.

8. C++ Implementation

Calendar I

```
#include <bits/stdc++.h>
using namespace std;

class MyCalendarI {
private:
    vector<pair<int, int>> bookings;

public:
    bool book(int start, int end) {
        for (auto& [s, e] : bookings) {
            if (start < e && end > s) {
                return false; // Overlap found
            }
        }
    }
}
```

```

        bookings.push_back({start, end});
        return true;
    }
};

```

Calendar II

```

class MyCalendarII {
private:
    vector<pair<int, int>> bookings;
    vector<pair<int, int>> doubleBookings;

public:
    bool book(int start, int end) {
        // Check against double-booked intervals
        for (auto& [s, e] : doubleBookings) {
            if (start < e && end > s) {
                return false; // Would cause triple booking
            }
        }

        // Add overlaps with existing bookings to doubleBookings
        for (auto& [s, e] : bookings) {
            if (start < e && end > s) {
                int overlapStart = max(start, s);
                int overlapEnd = min(end, e);
                doubleBookings.push_back({overlapStart, overlapEnd});
            }
        }

        bookings.push_back({start, end});
        return true;
    }
};

```

Calendar III

```

class MyCalendarIII {
private:
    map<int, int> events; // time -> delta (sorted)

public:
    int book(int start, int end) {
        events[start]++;
        events[end]--;

        int count = 0, maxCount = 0;
        for (auto& [time, delta] : events) {
            count += delta;

```

```

        maxCount = max(maxCount, count);
    }

    return maxCount;
}
};

```

Example Usage

```

int main() {
    // Calendar I
    MyCalendarI cal1;
    cout << boolalpha;
    cout << cal1.book(10, 20) << endl; // true
    cout << cal1.book(15, 25) << endl; // false
    cout << cal1.book(20, 30) << endl; // true

    // Calendar III
    MyCalendarIII cal3;
    cout << cal3.book(10, 20) << endl; // 1
    cout << cal3.book(50, 60) << endl; // 1
    cout << cal3.book(10, 40) << endl; // 2
    cout << cal3.book(5, 15) << endl; // 3
    cout << cal3.book(5, 10) << endl; // 3
    cout << cal3.book(25, 55) << endl; // 3

    return 0;
}

```

9. Python Implementation

```

class MyCalendarI:
    def __init__(self):
        self.bookings = []

    def book(self, start: int, end: int) -> bool:
        for s, e in self.bookings:
            if start < e and end > s:
                return False
        self.bookings.append([start, end])
        return True

class MyCalendarII:
    def __init__(self):
        self.bookings = []
        self.double_bookings = []

    def book(self, start: int, end: int) -> bool:

```

```

        # Check against double-booked intervals
        for s, e in self.double_bookings:
            if start < e and end > s:
                return False

        # Add overlaps with existing bookings to double_bookings
        for s, e in self.bookings:
            if start < e and end > s:
                overlap_start = max(start, s)
                overlap_end = min(end, e)
                self.double_bookings.append([overlap_start, overlap_end])

        self.bookings.append([start, end])
        return True

class MyCalendarIII:
    def __init__(self):
        self.events = {} # time -> delta

    def book(self, start: int, end: int) -> int:
        self.events[start] = self.events.get(start, 0) + 1
        self.events[end] = self.events.get(end, 0) - 1

        count = 0
        max_count = 0

        for time in sorted(self.events.keys()):
            count += self.events[time]
            max_count = max(max_count, count)

        return max_count

# Example usage
if __name__ == "__main__":
    cal3 = MyCalendarIII()
    print(cal3.book(10, 20)) # 1
    print(cal3.book(50, 60)) # 1
    print(cal3.book(10, 40)) # 2

```

10. Code Explanation

Calendar I

- **Simple check:** For each new booking, check against all existing bookings.
- **Overlap condition:** `start < existing_end && end > existing_start` — standard interval overlap.
- **O(n)** per booking.

Calendar II

- **Two lists:** `bookings` (all intervals) and `double_bookings` (overlaps between pairs).
- **Triple booking check:** If new interval overlaps with any `double_bookings`, it would create a third overlap → reject.
- **Update `double_bookings`:** For each existing booking, if they overlap, the overlap region becomes double-booked.
- **$O(n)$** per booking.

Calendar III

- **Sweep line:** `map` stores `time` → `delta`. Using `map` ensures sorted order.
- **Each booking:** Add +1 at start, -1 at end.
- **Query:** Iterate map, compute running count, track max.
- **$O(n \log n)$** per booking (n = number of events in map).

11. Complexity Analysis

Variant	Time per Booking	Space
Calendar I	$O(n)$	$O(n)$
Calendar II	$O(n)$	$O(n)$
Calendar III	$O(n \log n)$	$O(n)$

- **Calendar I/II:** $O(n)$ per booking where n is the number of existing bookings.
- **Calendar III:** $O(n \log n)$ per booking due to sorting the map keys. Can be optimized to $O(n)$ by using a balanced BST or a list that maintains sorted order.

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
No overlap	"No double booking"	Check all existing	My Calendar I
Limit on overlap	"No triple booking"	Track double-booked regions	My Calendar II
Max concurrent	"Maximum k-booking"	Sweep line	My Calendar III
Range module	"Add/remove/query ranges"	Set of intervals	Range Module

13. Common Mistakes

- **Calendar I — $O(n^2)$ per booking:** Using nested loops inefficiently. The naive approach is $O(n)$ per booking, which is fine.
- **Calendar II — Wrong overlap detection:** The condition `start < e && end > s` is correct only if you check both. Using just `start < e` is insufficient.
- **Calendar III — Not using `map`:** Using `unordered_map` loses the sorted order needed for sweep line.
- **Calendar III — Recomputing from scratch:** The `map` approach recomputes the max each time. For large n , this is $O(n \log n)$ per query. Use a segment tree for $O(\log n)$ per query.

14. Edge Cases

Variant	Input	Expected	Notes
Calendar I	[1,2], [2,3]	Both true	Touching is allowed
Calendar I	[1,5], [2,3]	Second false	Nested
Calendar II	[1,3], [2,4], [3,5]	All true	No triple overlap
Calendar II	[1,5], [2,3], [2,4]	Third false	Triple at [2,3]
Calendar III	[1,2], [2,3], [3,4]	Max = 1	No overlap

15. Variations

15.1 Range Module

Supports add, remove, and query operations on ranges. Maintain a set of non-overlapping intervals. Add merges, remove splits, query checks if a range is fully covered.

15.2 Calendar with Binary Search

For Calendar I, use a set of intervals sorted by start, and use binary search ([lower_bound](#)) to find the potential overlap in $O(\log n)$ time.

15.3 Calendar with Segment Tree

For Calendar III, use a segment tree with lazy propagation to get $O(\log n)$ per booking.

16. Related Algorithms/Data Structures

Algorithm	Connection	When to Choose
Sweep Line	Calendar III is pure sweep line	For max overlap queries
Interval Tree	Dynamic overlap queries	For Calendar I with many bookings
Segment Tree	Range updates and queries	For Calendar III with many bookings
Merge Intervals	Merges overlapping intervals	For Range Module

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
My Calendar I	LeetCode 729	No double booking	Medium
Meeting Rooms	LeetCode 252	Simple overlap check	Easy

Medium

Problem	Platform	Pattern	Difficulty
My Calendar II	LeetCode 731	No triple booking	Medium
My Calendar III	LeetCode 732	Max k-booking	Hard

Hard

Problem	Platform	Pattern	Difficulty
Range Module	LeetCode 715	Add/remove/query	Hard
Data Stream as Disjoint Intervals	LeetCode 352	Dynamic intervals	Hard

18. Interview Explanation

"Calendar Booking problems come in three variants. Calendar I checks for any overlap: I maintain a list of bookings and check each new one against all existing ones. Calendar II prevents triple booking: I maintain both a list of bookings and a list of double-booked regions; a new booking must not overlap with any double-booked region. Calendar III returns the maximum concurrent bookings: I use a sweep line with a map of time-to-delta events, processing them in sorted order to track the running count. Calendar I and II are $O(n)$ per booking, Calendar III is $O(n \log n)$."

19. Revision Notes

- **Calendar I:** Check `start < e && end > s` for each existing.
- **Calendar II:** Track double-booked regions; check against them.
- **Calendar III:** Sweep line with `map<int, int>` (+1/-1).
- **Touching is allowed:** `[1,2]` and `[2,3]` are fine.
- **Complexity:** $O(n)$ per booking for I/II, $O(n \log n)$ for III.

20. Final Cheat Sheet

Aspect	Details
When to use	Booking systems, overlap limits, max concurrent
Main operations	Check overlap (I), track double-booked (II), sweep line (III)
Time	$O(n)$ per booking (I/II), $O(n \log n)$ (III)
Space	$O(n)$
Key code	I: <code>if (s < e2 && e > s2) return false;</code>
	II: Check double_booked, add overlaps to double_booked
	III: <code>events[start]++; events[end]--;</code>
Edge cases	Touching allowed, triple overlap, empty calendar
Common trap	Wrong overlap condition, not using <code>map</code> for III

SEGMENT TREE WITH SWEEP LINE

1. Overview

Segment Tree with Sweep Line combines the segment tree data structure (for range updates and range queries) with the sweep line paradigm (processing events in order). This powerful combination is used for problems like "The Skyline Problem" and "Maximum Overlap Rectangle" where you need to process events and query/update ranges efficiently.

The segment tree handles the "vertical" dimension (range updates/queries) while the sweep line handles the "horizontal" dimension (processing events in order).

2. Intuition

Imagine you have rectangles on a 2D plane. You want to find the total area of their union. Or you want to find the maximum overlap. Or you want to draw the skyline.

Analogy: Think of a city skyline. Buildings (rectangles) are placed along a street. You sweep from left to right. When you encounter the left edge of a building, it "starts" and its height becomes active. When you encounter the right edge, it "stops". At any point, the current skyline height is the maximum active height. A segment tree helps you quickly find this maximum height and update it as buildings start/stop.

Step-by-step reasoning:

1. Collect all unique x-coordinates (start and end of each interval/rectangle).
2. Sort these x-coordinates (coordinate compression).
3. Build a segment tree over the y-dimension (or height dimension).
4. Process events sorted by x:
 - At a start event, add the interval/rectangle to the segment tree.
 - At an end event, remove it.
5. Use the segment tree to query the current state (max height, covered length, etc.).

3. When to Use It

Situation	Example Problem Phrase
2D rectangle overlap / union / intersection	"Area of rectangle union", "skyline"
Dynamic intervals with range updates and queries	"Add/remove interval, query max overlap"
Problems requiring both sweep and range operations	"Sweep line with segment tree"
Finding the outline of overlapping rectangles	"Skyline", "building outline"
Maximum overlap of weighted intervals	"Maximum overlapping weight"

Trigger phrases: "skyline", "rectangle union", "rectangle area", "segment tree sweep line", "interval tree 2D"

4. When Not to Use It

- **When only 1D intervals are involved:** Simple sweep line (without segment tree) is sufficient for 1D interval overlap.
- **When the coordinate space is small:** Use a simple array with difference array technique.
- **When you don't need range queries during sweep:** If you just need the max count, a simple sweep line with events is enough.
- **When the problem is simple enough for a balanced BST:** For problems like "max overlapping intervals" in 1D, a multiset is simpler.
- **For competitive programming time constraints:** Segment tree with sweep line is complex to implement. Use simpler approaches when possible.

5. Core Concepts

5.1 Coordinate Compression

Map large coordinates to smaller indices. Since segment tree size depends on the number of unique coordinates, not the coordinate range, compression is essential.

5.2 Segment Tree with Range Update

For sweep line problems, we often need to add/remove values over a range (e.g., add height to a range of y-coordinates). This requires a segment tree with lazy propagation for range updates.

5.3 Event Processing

Events are sorted by x-coordinate. For each x, we process all events at that x before moving to the next x.

5.4 Query After Each Sweep Step

After processing events at a given x, we query the segment tree for the current state (max height, covered length, etc.).

6. Step-by-Step Algorithm

Rectangle Area Union (2D Sweep Line + Segment Tree)

```
Input: rectangles [(x1, y1, x2, y2), ...]
```

1. Create events: for each rectangle:
 - (x1, y1, y2, +1) // left edge – start
 - (x2, y1, y2, -1) // right edge – end
2. Collect all unique y-coordinates, compress them.
3. Build segment tree over compressed y-coordinates.
 - Each node stores: coveredLength, count (how many rectangles cover this segment)
4. Sort events by x.
5. For each group of events at the same x:

```

a. For each event (y1, y2, delta):
    Update segment tree: add delta to range [y1, y2)
b. If not the last event:
    currentX = this event's x
    nextX = next event's x
    width = nextX - currentX
    height = segmentTree.queryCoveredLength()
    area += width * height

```

6. Return area.

7. Dry Run

For a complete dry run of a segment tree with sweep line (rectangle area union), the explanation is extensive. The key steps are:

Input rectangles: $(0, 0, 2, 2)$, $(1, 1, 3, 3)$

Y-coordinates: $0, 1, 2, 3 \rightarrow$ compressed to $0, 1, 2, 3$

Events:

- $(0, 0, 2, +1)$ — left edge of first rectangle
- $(1, 1, 3, +1)$ — left edge of second rectangle
- $(2, 0, 2, -1)$ — right edge of first rectangle
- $(3, 1, 3, -1)$ — right edge of second rectangle

Processing:

- At $x=0$: Add y-range $[0, 2) \rightarrow$ covered length = 2. Next $x=1$, width=1. Area += $1 \times 2 = 2$.
- At $x=1$: Add y-range $[1, 3) \rightarrow$ covered length = 3 (union of $[0, 2)$ and $[1, 3)$). Next $x=2$, width=1. Area += $1 \times 3 = 3$.
- At $x=2$: Remove y-range $[0, 2) \rightarrow$ covered length = 1 (only $[1, 3)$ remains). Next $x=3$, width=1. Area += $1 \times 1 = 1$.
- At $x=3$: Remove y-range $[1, 3) \rightarrow$ covered length = 0.

Total area: $2 + 3 + 1 = 6$

8. C++ Implementation

Here's a complete implementation for Rectangle Area Union using sweep line + segment tree.

```

#include <bits/stdc++.h>
using namespace std;

class SegmentTree {
private:
    struct Node {
        int count;        // number of active rectangles covering this segment
        long long len;    // covered length in this segment
    };

```

```

};

vector<Node> tree;
vector<int> ys; // compressed y-coordinates

void build(int node, int l, int r) {
    tree[node] = {0, 0};
    if (l == r) return;
    int mid = (l + r) / 2;
    build(node * 2, l, mid);
    build(node * 2 + 1, mid + 1, r);
}

void update(int node, int l, int r, int ql, int qr, int val) {
    if (ql > r || qr < l) return;
    if (ql <= l && r <= qr) {
        tree[node].count += val;
    } else {
        int mid = (l + r) / 2;
        update(node * 2, l, mid, ql, qr, val);
        update(node * 2 + 1, mid + 1, r, ql, qr, val);
    }

    // Update covered length
    if (tree[node].count > 0) {
        // Fully covered
        tree[node].len = ys[r + 1] - ys[l];
    } else if (l == r) {
        tree[node].len = 0;
    } else {
        tree[node].len = tree[node * 2].len + tree[node * 2 + 1].len;
    }
}

public:
    SegmentTree(vector<int>& yCoords) : ys(yCoords) {
        int n = ys.size() - 1; // number of segments
        tree.resize(4 * n + 5);
        if (n > 0) build(1, 0, n - 1);
    }

    void addRange(int y1, int y2, int val) {
        int l = lower_bound(ys.begin(), ys.end(), y1) - ys.begin();
        int r = lower_bound(ys.begin(), ys.end(), y2) - ys.begin() - 1;
        if (l <= r) update(1, 0, ys.size() - 2, l, r, val);
    }

    long long getCoveredLength() {
        return tree[1].len;
    }
};

long long rectangleAreaUnion(vector<vector<int>>& rectangles) {
    if (rectangles.empty()) return 0;

```

```

// Create events and collect y-coordinates
struct Event {
    int x, y1, y2, delta;
};
vector<Event> events;
vector<int> ys;

for (auto& rect : rectangles) {
    int x1 = rect[0], y1 = rect[1], x2 = rect[2], y2 = rect[3];
    events.push_back({x1, y1, y2, 1});
    events.push_back({x2, y1, y2, -1});
    ys.push_back(y1);
    ys.push_back(y2);
}

// Coordinate compression
sort(ys.begin(), ys.end());
ys.erase(unique(ys.begin(), ys.end()), ys.end());

// Sort events by x
sort(events.begin(), events.end(),
      [](const Event& a, const Event& b) { return a.x < b.x; });

// Build segment tree
SegmentTree segTree(ys);

long long area = 0;
int prevX = events[0].x;

for (int i = 0; i < (int)events.size(); i++) {
    // Add area from prevX to current x
    if (i > 0 && events[i].x > prevX) {
        long long width = events[i].x - prevX;
        long long height = segTree.getCoveredLength();
        area += width * height;
    }

    // Process current event
    segTree.addRange(events[i].y1, events[i].y2, events[i].delta);
    prevX = events[i].x;
}

return area;
}

// Example usage
int main() {
    vector<vector<int>> rectangles = {{0,0,2,2}, {1,1,3,3}};
    cout << "Rectangle area union: " << rectangleAreaUnion(rectangles) << endl;
    // Output: 7 (total area = 2*2 + 2*2 - 1*1 = 7)
}

```

```

    return 0;
}

```

9. Python Implementation

```

class SegmentTree:
    def __init__(self, ys):
        """ys: sorted unique y-coordinates"""
        self.ys = ys
        self.n = len(ys) - 1 # number of segments
        if self.n > 0:
            self.count = [0] * (4 * self.n)
            self.length = [0] * (4 * self.n)

    def _update(self, node, l, r, ql, qr, val):
        if ql > r or qr < l:
            return
        if ql <= l and r <= qr:
            self.count[node] += val
        else:
            mid = (l + r) // 2
            self._update(node * 2, l, mid, ql, qr, val)
            self._update(node * 2 + 1, mid + 1, r, ql, qr, val)

        # Update length
        if self.count[node] > 0:
            self.length[node] = self.ys[r + 1] - self.ys[l]
        elif l == r:
            self.length[node] = 0
        else:
            self.length[node] = self.length[node * 2] + self.length[node * 2 + 1]

    def add_range(self, y1, y2, val):
        l = bisect_left(self.ys, y1)
        r = bisect_left(self.ys, y2) - 1
        if l <= r:
            self._update(1, 0, self.n - 1, l, r, val)

    def get_covered_length(self):
        return self.length[1]

from bisect import bisect_left

def rectangle_area_union(rectangles):
    if not rectangles:
        return 0

    events = [] # (x, y1, y2, delta)
    ys = set()

```

```

for x1, y1, x2, y2 in rectangles:
    events.append((x1, y1, y2, 1))
    events.append((x2, y1, y2, -1))
    ys.add(y1)
    ys.add(y2)

ys = sorted(ys)
events.sort(key=lambda e: e[0])

seg_tree = SegmentTree(ys)

area = 0
prev_x = events[0][0]

for i, (x, y1, y2, delta) in enumerate(events):
    if i > 0 and x > prev_x:
        width = x - prev_x
        height = seg_tree.get_covered_length()
        area += width * height

    seg_tree.add_range(y1, y2, delta)
    prev_x = x

return area

# Example usage
if __name__ == "__main__":
    rects = [[0, 0, 2, 2], [1, 1, 3, 3]]
    print(f"Rectangle area union: {rectangle_area_union(rects)}")
# Output: 7

```

10. Code Explanation

- **Coordinate compression:** Unique y-coordinates are collected and sorted. The segment tree works on indices of these compressed coordinates.
- **Segment tree structure:** Each node stores `count` (how many rectangles cover this segment) and `len` (the actual covered length in original coordinates).
- **Update:** When adding/removing a rectangle, we update the range `[y1_idx, y2_idx - 1]` (since y2 is exclusive). The `count` is incremented/decremented.
- **Length computation:** If `count > 0`, the segment is fully covered $\rightarrow len = ys[r+1] - ys[l]$. Otherwise, `len = left.len + right.len`.
- **Sweep loop:** Events are grouped by x. For each group, first compute area for the previous segment (`width × current covered height`), then process all events at this x.

11. Complexity Analysis

Operation	Time	Space
Event creation	$O(n)$	$O(n)$

Operation	Time	Space
Coordinate compression	$O(n \log n)$	$O(n)$
Sorting events	$O(n \log n)$	$O(\log n)$
Segment tree update (per event)	$O(\log n)$	$O(\log n)$
Segment tree query	$O(1)$	$O(1)$
Overall	$O(n \log n)$	$O(n)$

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Rectangle area union	Find total area covered by rectangles	Sweep + segment tree for covered length	Rectangle Area (LC 850)
Skyline	Find outline of overlapping rectangles	Sweep + multiset for max height	The Skyline Problem (LC 218)
Rectangle overlap	Find if any two rectangles overlap	Sweep + interval tree	Rectangle Overlap
Max overlap rectangle	Find point with max rectangle overlap	Sweep + segment tree for max count	Maximum Overlap in 2D

13. Common Mistakes

- **Segment tree off-by-one:** The segment tree typically works on segments (between y-coordinates), not on points. $[y1, y2)$ means the covered range is $y1$ to $y2$ exclusive.
- **Coordinate compression errors:** Using `lower_bound` correctly for both left and right boundaries is critical.
- **Not handling multiple events at the same x:** All events at the same x must be processed before computing area for the next segment.
- **Segment tree lazy propagation:** For more complex operations (like max height), you need lazy propagation for range updates.
- **Integer overflow:** Use `long long` for area calculations.

14. Edge Cases

Input	Expected	Notes
<code>[]</code>	0	Empty
Single rectangle	<code>width * height</code>	Basic case
Non-overlapping rectangles	Sum of areas	Simple
Fully overlapping rectangles	Area of overlap	Merge
Rectangles with same x-coordinates	Correct area	Multiple events at same x

Input	Expected	Notes
Large coordinates	Works with compression	Coordinate compression handles this

15. Variations

15.1 Skyline Problem

Instead of computing union area, find the outline of overlapping rectangles. Use a multiset to track maximum active height. No segment tree needed — just a sweep line with a multiset.

15.2 Maximum Overlap in 2D

Find the point with the maximum number of overlapping rectangles. Use sweep line + segment tree with range updates (+1/-1) and query for max.

15.3 Rectangle Union Perimeter

Similar to area, but compute the perimeter of the union.

15.4 Count of Overlapping Rectangles

For each point, count how many rectangles cover it. Use 2D difference array if coordinates are small, or sweep line + segment tree.

16. Related Algorithms/Data Structures

Algorithm	Connection	When to Choose
Segment Tree	Core data structure for range operations	When you need range updates and queries
Sweep Line	Processing events in order	When you need to process interval starts/ends
Fenwick Tree (BIT)	Simpler for point updates, range queries	When you only need point updates
Interval Tree	Dynamic interval queries	For dynamic insertion/deletion
Coordinate Compression	Reduces coordinate space	Always needed with segment tree on large ranges

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
The Skyline Problem	LeetCode 218	Sweep + multiset	Hard
Rectangle Area	LeetCode 223	Simple overlap	Medium

Medium

Problem	Platform	Pattern	Difficulty
Rectangle Area II	LeetCode 850	Sweep + segment tree	Hard
My Calendar III	LeetCode 732	Sweep line	Hard

Hard

Problem	Platform	Pattern	Difficulty
Perfect Rectangle	LeetCode 391	Sweep + verification	Hard
Maximum Number of Visible Points	LeetCode 1610	Sweep + sliding window	Hard
Number of Islands II	LeetCode 305	DSU + sweep	Hard

18. Interview Explanation

"Segment Tree with Sweep Line is a powerful technique for 2D interval problems like rectangle area union. The sweep line processes events (rectangle edges) sorted by x-coordinate. The segment tree maintains the current state along the y-axis — typically the covered length or maximum height. Each time we cross a new x, we compute the area contribution using the current covered length. Coordinate compression is used to keep the segment tree size manageable. This gives $O(n \log n)$ time complexity."

19. Revision Notes

- **Sweep + segment tree** = 2D interval processing.
- **Coordinate compression** — essential for large coordinates.
- **Segment tree** maintains covered length or max height.
- **Events:** $(x, y1, y2, +1/-1)$.
- **Area** = **width** × **height** at each sweep step.
- **Complexity:** $O(n \log n)$.
- **Common use:** Rectangle area union, skyline.

20. Final Cheat Sheet

Aspect	Details
When to use	2D rectangle problems, area union, skyline
Main operations	Sweep by x, segment tree on y for range updates
Time	$O(n \log n)$
Space	$O(n)$
Key code	Events: $(x, y1, y2, \text{delta})$; seg tree: range add + query
Edge cases	Empty, single rect, overlapping, same x events
Common trap	Off-by-one in segment tree, coordinate compression errors

FINAL REVISION TABLE

Algorithm	Use Case	Time	Space	Key Idea
Merge Intervals	Merge overlapping intervals	$O(n \log n)$	$O(n)$	Sort by start, merge adjacent
Insert Interval	Insert into sorted non-overlapping list	$O(n)$	$O(n)$	Three phases: before, merge, after
Non-overlapping Intervals	Max non-overlapping (min removals)	$O(n \log n)$	$O(1)$	Sort by end, greedy keep
Meeting Rooms	Check if any overlap exists	$O(n \log n)$	$O(1)$	Sort by start, adjacent check
Minimum Platforms	Min rooms/platforms needed	$O(n \log n)$	$O(1)$	Two-pointer on sorted arrivals/departures
Sweep Line	General overlap counting	$O(n \log n)$	$O(n)$	Events (+1/-1), sort, process
Difference Array	Range add → final array	$O(n + m)$	$O(n)$	<code>diff[l]+=val, diff[r+1]-=val</code> , prefix sum
Line Sweep Sorting	Sort-based interval processing	$O(n \log n)$	$O(1)$	Sort first, then one pass
Calendar Booking	Booking with overlap limits	$O(n)$ per op	$O(n)$	I: check all; II: track double; III: sweep
Segment Tree + Sweep	2D rectangle problems	$O(n \log n)$	$O(n)$	Sweep by x, seg tree on y